UFS on MT6991: the bug, the fix, and the G5 puzzle

Why gear and lane changes failed, why one line fixed them, and what we've learned since about G5 speed. Explained from scratch.

Written 1 November 2026 · MediaTek MT6991 SoC evaluation board (`k6991v1_64`) · Android GKI kernel 6.6 (`android15-6.6`) with MediaTek vendor modules · Stands on its own: the earlier write-up isn't needed

THE WHOLE STORY ON ONE PAGE

1. The board's kernel has a debugfs file, `power_mode`. Writing six digits to it (for example `552211`) sets the gear, lanes and mode of the link between the processor and its UFS storage chip.
2. It failed whenever the board had been quiet for more than about half a second: first with a PMC timeout, then with `exit h8 state fail`.
3. The cause: the first line woke the UFS *controller* but not the *chip side*, which Linux represents as the W-LUN. Since Linux 5.14 those two wake separately. After half a second of quiet the chip is in Sleep and the link is in software hibern8, and only the W-LUN's wake-up undoes that.
4. The fix was one call: `ufs_debugfs_get_user_access()`, which runs `ufshcd_rpm_get_sync()` and wakes the W-LUN (and the controller first). On the board: no more failures, lane changes work, and fio speeds follow the gear.
5. Then a new question: G4 and G1 give steady numbers, but G5 writes to `/data` wander between about 3000 and 3700 MB/s.
6. Why only G5: every write passes through the CPU, an encryption engine, the controller, the link and the chip, and the slowest of them sets the speed. Up to G4 the link is the slowest by far. At G5 it no longer is, so one of the others shows through.
7. The new finding: the controller's AES encryption engine has its own clock, 416 or 546 MHz, and only MediaTek's own helper ever changes it. Gear changes through `power_mode` don't. During the steady-3000 run it sat at 416 MHz.
8. Also measured: the controller's main clock drops to 273 MHz only while the disk sleeps at G5 (MediaTek's sleep and wake hooks), and SQPOLL in fio raised a 4 GB run from 3200 to 3800 MB/s, so the CPU can be a limit too.
9. Not yet proven: that the slow encryption clock is what holds G5 writes near 3000. One clean run settles it (4.11).
10. Recommended now: drop the ported clock-scaling block (file 07), and set MediaTek's clocks fast and locked once per boot (`echo 5`, then `echo 4`, to `clkscale_control`).

How to read this

Every idea starts with the physical thing (wires, chips, registers, clocks) before it gets a name. Because this document mixes things we measured with things we read in the code and things we only suspect, every important claim carries one of three labels:

SEEN ON THE BOARD — Measured on the board: a screenshot, a dmesg line, a benchmark.

FROM THE CODE — I read it in the source copies in `sources/`, with the file and line.

NOT YET PROVEN — A hypothesis. The test that would settle it is always next to it.

And six kinds of boxes:

PLAIN WORDS — The simplest version of the idea.

TWO MEANINGS — A word that means different things in different places.

KEY POINT — The thing to remember from that section.

WATCH OUT — A trap you can fall into on the board.

CORRECTION — Something stated earlier in the investigation and later corrected. Part 5 lists them all.

TRY IT — Commands to run on the board.

Line numbers refer to the source copies in `sources/` next to this file: the Android common kernel `android15-6.6` (the UFS core), MediaTek's vendor code from OnePlus's public MT6991 tree, upstream Linux v5.13, and Qualcomm's msm-4.19 debugfs code. The board's vendor tree has the same code but may be a few lines off, and its `ufshcd.c` carries the port's changes, so search by function name.

Contents

1. The bug 3
2. How it was found, and what changed 9
3. Why it happened 16
4. What we have now 29
5. Corrections along the way 40
6. Command cheat sheet 42
7. Glossary 44
8. Files and sources 48
9. References 49

1 The bug

1.1 What was being built

The goal: change the UFS link's gear and lane count at runtime on the MT6991 evaluation board, so the storage can be tested at each speed.

MediaTek's driver already offers a control file, `clkscale_control` (section 4.3), but it only knows two settings, "fast" and "slow". The goal needed any gear and any lane count, so a debugfs file, `power_mode`, was added to the UFS core's `drivers/ufs/core/ufs-debugfs.c`. The code was ported from the Qualcomm SoC kernel, msm-4.19 (`ufsdbg_config_pwr_mode()`, file 04 in this folder).

You write six digits, `GGLLMM`: gear RX, gear TX, lanes RX, lanes TX, mode RX, mode TX. Mode 1 is FAST (the high-speed HS gears) and 2 is SLOW (the slow PWM gears). So:

YOU WRITE	MEANING
`552211`	HS-G5, two lanes each way, FAST mode
`442211`	HS-G4, two lanes, FAST
`551111`	HS-G5, one lane each way, FAST
`112211`	HS-G1, two lanes, FAST

The mode digits have two more values: `4` is FASTAUTO and `5` is SLOWAUTO, the same speeds except that the link may drop into a lower-power state on its own between bursts of data. MediaTek's negotiation doesn't accept them, so the ported function maps them to FAST and SLOW (2.3).

> **TWO MEANINGS**
>
> **"Two lanes"** means two *per direction*: 2 TX and 2 RX, four wire pairs. In the six digits, the lane part `22` is one digit for RX and one for TX.
>
> **"Power mode"** has three meanings in this story, and the bug involved all three at once:
>
> 1. **Link power mode** (UniPro): FAST, SLOW, FASTAUTO, SLOWAUTO, the link's speed setting. The digits set this.
> 2. **Chip power mode**: Active, Sleep, PowerDown, set by sending the chip the SCSI command START STOP UNIT (SSU).
> 3. **Linux runtime PM state** of a `struct device`: active or suspended (3.2).

The write runs `ufsdbg_power_mode_write()`, which parses the digits, keeps the current rate (A or B), checks the request with `ufsdbg_power_mode_validate()`, compares it with the chip's maximum in `ufsdbg_cfg_pwr_param()`, and then calls `ufsdbg_config_pwr_mode()`, which carries out the power mode change (PMC).

The ported function, step by step

As of 31 October, the ported `ufsdbg_config_pwr_mode()` does this:

a. **Gets access to the hardware.** Originally with `pm_runtime_get_sync(hba->dev)`; now with `ufs_debugfs_get_user_access(hba)` (2.2).
b. **`ufshcd_hold()`** keeps the controller's clocks on. Apart from runtime PM, the core also switches the controller's clocks off during short idle gaps. This is called **clock gating**, and on MediaTek it kicks in after at least 10 ms. "Hold" blocks it while the code works.
c. **`down_write(&hba->clk_scaling_lock)`** keeps the core's clock scaling and its internal management commands (queries to the chip) from running at the same time. It's 6.6's replacement for msm-4.19's `hba->lock`. The error handler is held off by `host_sem`, which user access takes (2.2).
d. **`ufshcd_scsi_block_requests()`** tells the SCSI layer to stop sending new reads and writes.
e. **`ufshcd_wait_for_doorbell_clr()`** waits until every command already sent has finished, that is, until the doorbell register (0x58) reads all zeros.
f. **The ported clock lines** (4.10) and **`ufshcd_config_pwr_mode()`**, which runs MediaTek's `PRE_CHANGE` hook, then the PMC, then the `POST_CHANGE` hook (1.4).
g. **Undoes everything** in reverse order.

Where the ported code lives: core and vendor

On Android, Google builds one generic kernel image for every phone, the **GKI** (Generic Kernel Image). The UFS **core** is part of it: `drivers/ufs/core/ufshcd.c`, `ufs-debugfs.c` and their friends. MediaTek's UFS code (`ufs-mediatek.c`, `ufs-mediatek-sysfs.c`, …) is a **vendor module** loaded on top, from `kernel_device_modules`.

The core calls into the vendor module through **vops** ("variant operations"): a table of function pointers that MediaTek fills in. The ones in this story are:

- `pwr_change_notify`, called just before a PMC (`PRE_CHANGE`) and just after it (`POST_CHANGE`);
- `suspend` and `resume`, called when the chip side goes to sleep or wakes (3.3);
- `setup_clocks`, called whenever the core turns the controller's clocks on or off;
- `clk_scale_notify`, called around the core's clock scaling, which is off on the board (4.3).

The debugfs file lives in the core. It can call core functions, but it can't see MediaTek's private data (`struct ufs_mtk_host`) or call MediaTek's private functions. That's why the fix had to use a core helper, why the core can't move the encryption clock (4.4), and why a change to this file needs a new kernel image, not just a new vendor module.

> **TWO MEANINGS: DEBUGFS AND SYSFS**
>
> Both are files the kernel invents on the fly. **sysfs** (`/sys`) is the stable interface for real users; MediaTek's `clkscale_control` lives there. **debugfs** (`/sys/kernel/debug`) is for developers and can change at any time; the `power_mode` file lives there.

1.2 The two errors, and when they appeared

SEEN ON THE BOARD You hit two different messages, one per version of the ported function:

- **First version**, which called `ufshcd_change_power_mode()` directly, the way Qualcomm's code does: `pwr ctrl cmd 0x… with mode 0x… completion timeout`, after half a second. It was first reported as "pmc command timed out".

- **Second version**, which went through `ufshcd_config_pwr_mode()` so that MediaTek's hooks run: `exit h8 state fail`, followed by a full reset of the controller.

Both appeared only when the board had been quiet for more than about half a second. When you type commands into a shell, that's almost always. If something had just used the disk, the write could succeed, which is what made the bug so confusing. Part 3 explains why the half second matters.

1.3 The hardware, from scratch

Two chips and some wires. The UFS **controller** is a block inside the MT6991 processor (the SoC). The UFS **chip** is a separate package on the board: its own small controller plus the flash memory. Between them run the **lanes**: each lane is a pair of wires carrying data in one direction, so "two lanes" means two pairs each way, four pairs in all. The electrical standard for those wires is MIPI **M-PHY**.

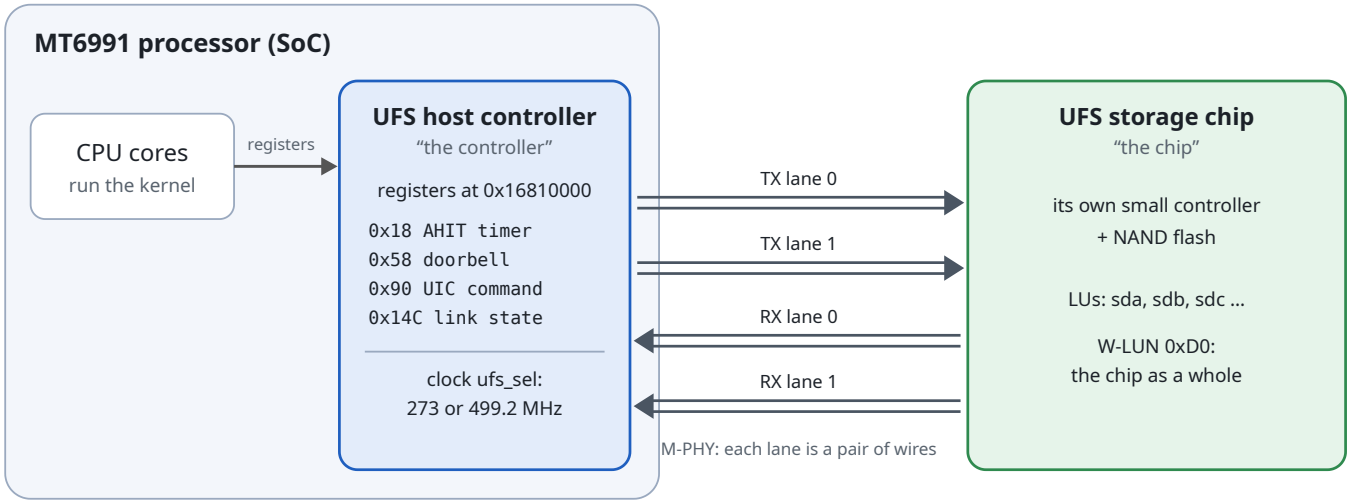


Figure 1. The controller is inside the SoC; the chip is a separate package. The CPU never touches the lanes. It only reads and writes the controller's registers, and the controller drives the wires.

The CPU never touches the wires. It reads and writes the controller's **registers**: memory addresses wired to hardware instead of RAM. Writing a number there makes the hardware do something; reading one tells you what it is doing. The controller's registers start at `0x16810000` (`mt6991.dts:6686`). Four of them matter in this story:

OFFSET	NAME	WHAT IT DOES
`0x18`	AHIT (auto-hibernate idle timer)	How long the link may sit idle before the controller puts it to sleep by itself. 0 turns the timer off (3.1).
`0x58`	Transfer request doorbell	One bit per command slot. The driver sets a bit to start a command; the controller clears it when the command is done.
`0x90`	UIC command	Commands for the link itself: wake it, put it to sleep, read or change a link setting.

OFFSET	NAME	WHAT IT DOES
`0x14C`	MediaTek link state (`REG_UFS_UFS_MMIO_OTSD_CTRL`)	MediaTek-specific. Its top byte says whether the link is up, down or asleep (3.1).

Three layers of rules on the wires

UFS transport (UTP)	Carries SCSI commands (READ, WRITE, START STOP UNIT) in packets called UPIUs. This is what the files are made of.
UniPro	The link layer: packs bits into frames, numbers them, confirms delivery, resends lost ones, and negotiates speed and sleep with the other end. Its lowest part is the PHY Adapter (PA); settings named `PA_…` live there.
M-PHY	The electrical layer: drives the voltages, recovers bits and timing at the other end, puts the wires to sleep and wakes them.

The chip runs the same three layers on its side. Every one of them has to agree before data moves.

> **PLAIN WORDS**
>
> UFS transport is the letter. UniPro is the post office: it numbers the letters, confirms delivery and re-sends lost ones. M-PHY is the road.

Gears: how fast the wires flip

A **gear** is how many bits per second each lane carries. The high-speed gears run from G1 to G5, and each doubles the one before. MediaTek uses "rate B", one of two slightly different base frequencies.

GEAR	BITS PER SECOND PER LANE	ONE BIT LASTS	CEILING, 2 LANES, BEFORE OVERHEAD
HS-G1	1,457.6 million	686 ps	~290 MB/s
HS-G2	2,915.2 million	343 ps	~580 MB/s
HS-G3	5,830.4 million	172 ps	~1.17 GB/s
HS-G4	11,660.8 million	86 ps	~2.3 GB/s
HS-G5	23,321.6 million	43 ps	~5.7 GB/s

G1 to G4 send every 8 bits of data as 10 bits on the wire (8b/10b), which keeps the signal balanced and lets the receiver stay locked to the timing. That's why the G1 to G4 ceilings are bit rate × 0.8 × lanes ÷ 8. G5 uses a more efficient code, so its ceiling is closer to the raw rate. Real throughput always lands below the ceiling because of protocol overhead. There is also a slow family, the PWM gears, used for bring-up and low power.

To make G5 physical: one bit lasts 43 picoseconds. In that time a signal on a circuit-board trace travels about 6 or 7 millimetres. That is why G5 leaves very little margin on an evaluation board with connectors and long

traces, and why G4 and G5 need the link to **train** itself (equalization, called ADAPT in UniPro) each time it enters those gears.

The clocks and supplies that matter later

FROM THE CODE Inside the controller, two clocks matter for speed. Both are multiplexers ("muxes") that pick one of two sources (`mt6991.dts:6700` to `6753`):

CLOCK	WHAT IT DRIVES	SLOW SOURCE	FAST SOURCE
`ufs_sel` (named `ck_sel` on the board)	The controller itself: how fast it moves data between memory and the link	273 MHz, `ck_mainpll_d4_d2`	499.2 MHz, `ck_univpll_d5`
`ufs_fde` (named `ck_aes_ufsfde_sel`)	The inline AES encryption engine (4.5)	416 MHz, `ck_univpll_d6`	546 MHz, `ck_mainpll_d4`

The fast `ufs_sel` source needs the SoC's core voltage, vcore, held at 0.65 V or more (`clk-scale-up-vcore-min = <650000>`, `mt6991.dts:6751`). The link itself has separate clocks that follow the gear: `unipro_tx_sym`, and one receive clock per lane, `unipro_rx_sym0` and `unipro_rx_sym1` (`mt6991.dts:6723`), because each lane's receiver recovers its timing from the bits arriving on it.

The chip runs on two supply rails, made by a separate power-management chip, the PMIC (an MT6363 here). Only the host switches them; the chip has no say:

- **VCC** feeds the flash memory (`vcc-supply = <&mt6363_vemc>`, `mt6991.dts:6752`).
- **VCCQ**, 1.2 V, feeds the chip's own controller and its end of the link (`vccq-supply = <&mt6363_vufs12>`, `mt6991.dts:6753`).

> TWO MEANINGS
>
> **"Host"** can mean the controller hardware, Linux's SCSI host object, or loosely the whole phone. **"Device"** can mean the UFS chip or any `struct device` in Linux's tree of objects. In this document, *the controller* and *the chip* always mean the hardware, and a plain "device" means a Linux object.

1.4 A power mode change, call by call

Changing gear, lanes or mode is a **power mode change** (PMC). Both ends of the link must switch at the same moment, so a PMC is a conversation:

1. The driver writes the new settings into the controller's UniPro attributes: `PA_RXGEAR`, `PA_TXGEAR`, the active lane counts, termination, the HS series (rate A or B), and some timeout values.
2. It writes `PA_PWRMODE`. That write is the "go".
3. UniPro sends a power-change request across the wires and waits for the chip's confirmation. UniPro calls these control frames `PACP_PWR_req` and `PACP_PWR_cnf`.
4. When the exchange finishes, whether it worked or not, the controller raises an interrupt with the result. "Local success" means done.

FROM THE CODE In the 6.6 core (`sources/android15-6.6/ufshcd.c`), with line numbers on the right:

```
ufshcd_config_pwr_mode()                 4652  the ported function calls this
 ├─ vendor hook, PRE_CHANGE              ufshcd-priv.h:182
 │    → MediaTek ufs_mtk_pwr_change_notify()  ufs-mediatek.c:2023
 │      saves AHIT, turns auto-hibern8 off, picks the final gear/lanes/mode
 └─ ufshcd_change_power_mode()           4560
     ├─ "power already configured": return, nothing sent   4566
     ├─ DME_SETs: PA_RXGEAR, PA_TXGEAR, lanes, termination, HS series,
     │   timeouts. Each goes ufshcd_dme_set_attr() 4074 → __ufshcd_send_uic_cmd() 2468
     ├─ ufshcd_uic_change_pwr_mode()     4333  writes PA_PWRMODE: the actual "go"
     │   └─ ufshcd_uic_pwr_ctrl()        4199  sends it, then sleeps on uic_async_done
     │        for up to 500 ms (UIC_CMD_TIMEOUT_DEFAULT, line 59).
     │        Woken by the interrupt handler ufshcd_uic_cmd_compl() 5472,
     │        which calls complete() at 5493 when the power-mode interrupt arrives.
     └─ only on success: vendor hook POST_CHANGE (MediaTek writes AHIT back),
        then hba->pwr_info = the new mode   4638
```

It can fail in three ways, and each prints something different:

MESSAGE	LINE	WHAT IT MEANS
`pwr ctrl cmd … uic error`	4229	The host's own UniPro refused the command. You see it at once.
`pwr ctrl cmd … failed, host upmcrs:…`	4254	The exchange finished, but the result wasn't "local success".
`pwr ctrl cmd … completion timeout`	4237	The exchange never finished: the host accepted the command and heard nothing back for 500 ms. This was the first error.

After any of them, unless a suspend or resume is in progress, the core marks the link broken and schedules the error handler (`ufshcd.c:4270` to `4272`). The error handler resets the controller and brings the link back up at its maximum gear, so the requested gear is lost.

Two more details from the same code. If you ask for exactly the mode the link already has, the core sends nothing and returns success (`ufshcd.c:4566`). And the core prints nothing when a PMC succeeds (`ufshcd_print_pwr_info()` uses `dev_dbg`, `ufshcd.c:697`). The `PMC Success!` lines you may see in dmesg come from MediaTek's own `clkscale_control` path, not from `power_mode`.

> **KEY POINT**
>
> A PMC is a conversation over the link. If the link is asleep, nobody answers.

2 How it was found, and what changed

2.1 The investigation, in order

1. **The setup.** MediaTek's driver offered only `clkscale_control`, so the debugfs `power_mode` from the Qualcomm SoC kernel was ported into the GKI core.
2. **Error 1, the PMC timeout.** The first version sent the PMC straight through `ufshcd_change_power_mode()`. After half a second without an answer, the core gave up (1.4).
3. **Error 2, "exit h8 state fail".** Routing the call through `ufshcd_config_pwr_mode()` made MediaTek's `PRE_CHANGE` hook run first. It tried to make sure the link was awake, waited 100 ms, and complained (`ufs_mtk_auto_hibern8_disable()`, `ufs-mediatek.c:2997`).
4. **Reading three code bases side by side.** Qualcomm's msm-4.19 debugfs code, where the first line came from. The 6.6 UFS core, where runtime power management lives today. MediaTek's vendor module, where the hook that complained lives.
5. **The key observation.** The first line, `pm_runtime_get_sync(hba->dev)`, wakes only the controller. Since Linux 5.14 the chip side, the W-LUN, has its own sleep and wake, and only its wake takes the link out of software hibern8 (Part 3).
6. **The fix, 31 October.** One call replaced: no more failures, and lane changes from 2 to 1 pass.
7. **1 November.** A review of the whole function, a clean version (file 07), and the G5 investigation that fills Part 4.

The path in more detail

1. **The question.** The starting point: the board, `clkscale_control`, the ported debugfs code and the two errors, and a search for a ready-made fix.
2. **The search.** We went through upstream Linux, MediaTek's vendor trees and the patch archives. No patch fixes this exact failure. Two are related, and neither fixes it: `7dec5cedf7b2` ("scsi: ufs: core: Serialize AHIT register access between sysfs and host driver") and `f29ec85ac4fd` ("scsi: ufs: host: mediatek: Return error directly on idle wait timeout"). We downloaded MediaTek's MT6991 vendor code (OnePlus's public copy), the android15-6.6 core files and Qualcomm's msm-4.19 `ufs-debugfs.c`, so the three could be read side by side. They are now in `sources/`.
3. **One step at a time.** The first explanation was too compressed, so we slowed down and went through it a question at a time: what disabling auto-hibern8 actually does, whether the link must be up before a PMC, why the W-LUN's suspend would have sent `DME_HIBERNATE_ENTER`, and why there's a separate step that powers UniPro down.
4. **Code to compare.** Files 01 to 05 were written into this folder, to open side by side in an editor.
5. **The flow.** The ported code's sequence: `pm_runtime_get_sync()`, hold, lock, block, doorbell, PMC, plus the ported clock lines. Its first step was the root cause: it woke `hba->dev`, not the W-LUN.
6. **The clock question.** Keep the ported clock-scaling lines, or copy MediaTek's? On 31 October we settled on keeping them, setting `clkscale_control` to 4 first, and never mixing with 1 or 3. On 1 November that changed: delete them (4.10).
7. **The ported code.** The code switched to `ufs_debugfs_get_user_access()` (from file 05) on 31 October. It was copied into file 06, with two more fixes marked.

8. **The test.** We rebuilt the kernel image, set `clkscale_control` to 4 and wrote `power_mode`: no `exit h8 state fail`. Then the lane tests passed too. The `echo 4` turned out not to be needed for the fix (4.3).

2.2 The one line that mattered

Before (as ported from Qualcomm msm-4.19)

```
pm_runtime_get_sync(hba->dev);
ufshcd_hold(hba);
...
/* block I/O, wait for the doorbell,
   then the PMC */
```

After (what runs now)

```
ret = ufs_debugfs_get_user_access(hba);
if (ret)
        return ret;
ufshcd_hold(hba);
...
ufs_debugfs_put_user_access(hba);
```

FROM THE CODE The new call is the UFS core's own helper, the one its other debugfs files already use (`ufs-debugfs.c:68`):

```
static int ufs_debugfs_get_user_access(struct ufs_hba *hba)
__acquires(&hba->host_sem)
{
        down(&hba->host_sem);                    // one big UFS job at a time
        if (!ufshcd_is_user_access_allowed(hba)) { // refuses during shutdown
                up(&hba->host_sem);
                return -EBUSY;
        }
        ufshcd_rpm_get_sync(hba);                 // wake the W-LUN
        return 0;
}
```

And `ufshcd_rpm_get_sync()` is a one-line wrapper (`ufshcd-priv.h:344`):

```
static inline int ufshcd_rpm_get_sync(struct ufs_hba *hba)
{
        return pm_runtime_get_sync(&hba->ufs_device_wlun->sdev_gendev);
}
```

It's the same `pm_runtime_get_sync()` as before. The only difference is *which device* it names: the W-LUN instead of `hba->dev`. Which device you name decides what wakes up. What now happens when you write `power_mode` after half a second of quiet:

1. The controller wakes first (Linux wakes a device's parents before the device): power domain, supplies, clocks, interrupt.
2. The W-LUN wakes: `DME_HIBERNATE_EXIT` (link UP), START STOP UNIT (chip Active), AHIT timer back on.
3. MediaTek's `PRE_CHANGE` hook turns AHIT off and checks 0x14C. The link is already UP, so the check passes at once.
4. The PMC runs over an awake link. `POST_CHANGE` puts AHIT back.
5. The code releases everything, and half a second later the chip side sleeps again.

Before the fix: `pm_runtime_get_sync(hba->dev)`

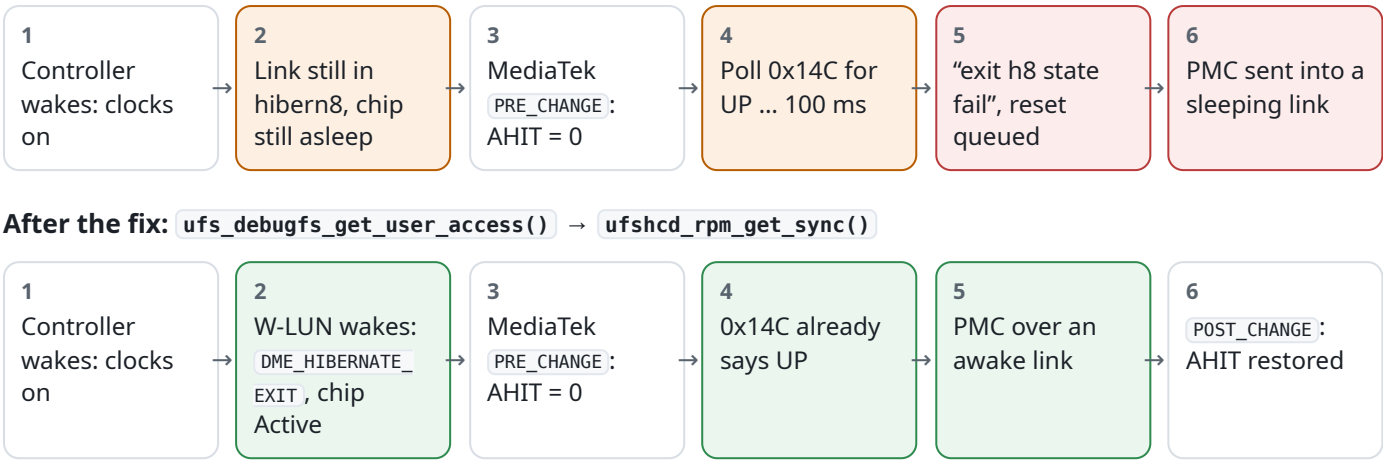


Figure 2. The same write, before and after. The only difference is step 2.

> **KEY POINT**
>
> MediaTek's own `clkscale_control` path never had this bug: it calls `ufshcd_rpm_get_sync()` before anything else (`ufs-mediatek.c:2322`). The fix makes the path follow the same recipe.

2.3 The other changes in the ported function

The mode digits: map AUTO, don't force FAST

```
        if (desired_pwr_mode->pwr_rx == FASTAUTO_MODE)
                desired_pwr_mode->pwr_rx = FAST_MODE;
        else if (desired_pwr_mode->pwr_rx == SLOWAUTO_MODE)
                desired_pwr_mode->pwr_rx = SLOW_MODE;
        desired_pwr_mode->pwr_tx = desired_pwr_mode->pwr_rx;
```

FROM THE CODE MediaTek's `ufs_mtk_pre_pwr_change()` (`ufs-mediatek.c:1908`) sets up a high-speed host by default and switches to PWM only if SLOW was requested. It then calls the shared negotiation helper `ufshcd_get_pwr_dev_param()` (in `ufshcd-pltfrm.c`, not copied here), which treats only FAST as "high speed". A FASTAUTO request therefore fails negotiation. MediaTek prints "failed to determine capabilities" and carries on with empty settings. Mapping the AUTO modes avoids that, and keeps SLOW requests as PWM. An early version of this block forced FAST for everything, which would have turned every PWM request into HS; that was wrong.

Don't report success after a queued reset

```
        if (!ret && hba->ufshcd_state != UFSHCD_STATE_OPERATIONAL)
                ret = -EIO;
```

MediaTek's hook can fail, queue a reset and still let the PMC report success. This turns that case into an error. The states (`ufshcd.h:529`):

VALUE	STATE	MEANING HERE
0	`UFSHCD_STATE_RESET`	Being reset

VALUE	STATE	MEANING HERE
1	`UFSHCD_STATE_OPERATIONAL`	All good
2	`UFSHCD_STATE_EH_SCHEDULED_NON_FATAL`	Error handler queued for a minor error
3	`UFSHCD_STATE_EH_SCHEDULED_FATAL`	Reset queued. This is what MediaTek's failed check sets.
4	`UFSHCD_STATE_ERROR`	Recovery failed

The validate function

Qualcomm's original `ufsdbg_power_mode_validate()` accepts gears up to G4 and only FAST or FASTAUTO, so it rejects G5 and every SLOW request. The version in file 06 accepts G1 to G5 and all four modes. `ufsdbg_cfg_pwr_param()` still refuses anything above the device's maximum gear or lane count, with the message "exceeds device limitation".

2.4 Is the whole function OK? A review

Yes: the part that matters is right, and not by luck.

- It does the same steps, in the same order, as the core does before its own gear changes (`ufshcd_clock_scaling_prepare()`, `ufshcd.c:1319`): wake, keep the clocks on, take `clk_scaling_lock`, stop new commands, wait for the ones in flight, then the PMC.
- The wake is the core's own helper.
- The PMC goes through `ufshcd_config_pwr_mode()`, so MediaTek's hooks run. Those hooks send only link commands (DME), nothing that needs `clk_scaling_lock`, so holding that lock for writing can't deadlock against them.

Three things still worth changing:

1. **Delete the ported clock-scaling block.** On the board it moves one clock in a way MediaTek's own code never does, and it can't touch the clock that matters for encrypted writes (4.10).
2. **Move the debug dumps.** If `ufshcd_dump_uic_state()` reads controller registers (its body wasn't available), two of its calls can run with the clocks off: the "before hold" one, and the one in the access-failure path. MediaTek turns the clocks off 10 ms after the last command (`clk_gating.delay_ms = max(ah_ms, 10U)`, `ufs-mediatek.c:1471`), but the W-LUN stays awake for about 500 ms. In that gap the wake does nothing and the clocks are off, and a register read there can hang the bus. Move the first call below `ufshcd_hold()`, and delete the failure-path one, which only runs during shutdown.
3. **Set MediaTek's clocks fast and locked once per boot** (`echo 5`, then `echo 4`; 4.3 and 4.12).

> **WATCH OUT**
>
> Any UFS error resets the link to maximum gear and lanes, without telling you. After each benchmark run, check `dmesg | grep "err_handler started"`. If that line appears, the run wasn't at the requested gear.

2.5 The clean version (file 07)

`07_clean_ufsdbg_config_pwr_mode.c` rewrites the ported function around one idea: the "stop I/O, then PMC" sequence belongs in the core, next to the static functions it uses. It has three parts, each labelled with where it goes. It hasn't been compiled yet.

Part A, `ufshcd.c`: one new core helper

```
int ufshcd_config_pwr_mode_quiesced(struct ufs_hba *hba,
                                    struct ufs_pa_layer_attr *pwr)
{
        int ret;

        ufshcd_hold(hba);                          // clocks on
        down_write(&hba->clk_scaling_lock);        // no queries, no scaling
        ufshcd_scsi_block_requests(hba);           // no new reads and writes

        ret = ufshcd_wait_for_doorbell_clr(hba, USEC_PER_SEC);
        if (!ret)
                ret = ufshcd_config_pwr_mode(hba, pwr); // vendor hooks run

        ufshcd_scsi_unblock_requests(hba);
        up_write(&hba->clk_scaling_lock);
        ufshcd_release(hba);
        return ret;
}
```

In stock 6.6, `ufshcd_scsi_block_requests()`, `ufshcd_scsi_unblock_requests()` and `ufshcd_wait_for_doorbell_clr()` are static in `ufshcd.c` (lines 378, 372 and 1210). The port made them, and `ufshcd_scale_clks()`, visible to `ufs-debugfs.c`. With the helper living in `ufshcd.c`, all four can go back to static. Grep your tree first, in case something else uses them. No export is needed: `ufs-debugfs.c` is built into the same module.

Part B, `ufshcd-priv.h`: its declaration

Part C, `ufs-debugfs.c`: the ported function and the write handler

- **Refuses with `-EBUSY`** while the core's automatic clock scaling is active, because its governor would undo the requested gear (it's off on the board, 4.3).
- **Maps the AUTO modes**, then takes user access (the W-LUN wake) and calls the helper.
- **Returns `-EIO`** if a reset was queued.
- **Returns `-ERANGE` if the vendor lowered the request without an error**, and says what the link really runs at. It compares `hba->pwr_info` with what was requested. This catches a G5 request quietly turned into G4.
- **Prints one `dev_info` line on success**, because the core prints nothing.
- **The write handler returns the real error** instead of `cnt`, so a failed `echo` really fails, and it checks for exactly six digits.
- **No clock block and no register dumps.**

TRY IT AFTER BUILDING

```
P=/sys/kernel/debug/ufshcd/16810000.ufshci/power_mode   # adjust if yours lives elsewhere
echo 112211 > $P; echo $?     # 0, dmesg: "power_mode: [RX, TX] gear [1, 1] ..."
echo 442211 > $P; echo $?     # 0, back to G4
echo 552211 > $P; echo $?     # 0 at G5, or an error that says why not
cat $P                        # what the core recorded (hba->pwr_info)
dmesg | grep "err_handler started"   # should print nothing
```

2.6 The `#if 0` manual hibern8 exit

The ported code has a disabled block that called `ufshcd_uic_hibern8_exit()` directly. It aimed at the right symptom, but it would only have woken the link:

- The chip would still be in Sleep from START STOP UNIT.
- The core's records would still say the W-LUN is suspended and the link is in hibern8, because `ufshcd_uic_hibern8_exit()` doesn't update `hba->uic_link_state`; its callers do (`ufshcd.c:4413`).
- The next real wake-up would then most likely send `DME_HIBERNATE_EXIT` to a link that's already awake, and fail.

Waking the W-LUN does all three things, in the order the core expects.

Qualcomm's original also enters and exits hibern8 by hand, but for a different reason: around its clock switch, so that auto-hibern8 can't race it (`ufs-debugfs.c:1237` in msm-4.19).

2.7 Testing the fix

SEEN ON THE BOARD Results so far: with the fix, no `exit h8 state fail`; lane changes from 2 to 1 pass; fio speeds follow the gear. These three tests turn that into proof.

Test 1: recreate the old failure condition

The old failure needed the W-LUN asleep. Anything that touches the disk wakes it for a moment (an `echo 4` does too), so a write right after it could pass even with the old code. Make sure the W-LUN is asleep before you write:

TRY IT

```
W=/sys/bus/scsi/devices/0:0:0:49488/power/runtime_status   # the UFS Device W-LUN
cat $W                  # repeat until it says: suspended
cat $W; echo 112211 > /sys/kernel/debug/…/power_mode; dmesg | tail
```

It passes if you see `suspended`, then no `exit h8 state fail`. If `0:0:0:49488` isn't there, find it with `ls /sys/bus/scsi/devices | grep 49488`.

Test 2: did the gear really change?

"No error" isn't proof, for four reasons:

1. **The write never fails in the shell.** The original `ufsdbg_power_mode_write()` ends with `return cnt;` even when the PMC failed, so `echo` always looks successful. Only dmesg tells you. File 07 changes that.
2. **The success line in dmesg isn't from `power_mode`.** `PMC Success! [RX, TX]: gear=…` comes from MediaTek's `ufs_mtk_config_pwr_mode()`, that is, from `clkscale_control`. The core prints nothing on success (`ufshcd_print_pwr_info()` uses `dev_dbg`, `ufshcd.c:697`).
3. **Asking for the current mode does nothing.** If the request equals the current settings, `ufshcd_change_power_mode()` returns 0 without sending a PMC ("power already configured", `ufshcd.c:4566`).
4. **`cat power_mode` shows what the core believes.** It prints `hba->pwr_info`, which the core updates only after a successful PMC (`ufshcd.c:4638`). That's good evidence, but it's still software reporting on itself.

The physical check: the wire sets a speed limit. Whatever the software says, data can't move faster than the lanes carry it. The ceilings are in the gear table in 1.3: about 290 MB/s for HS-G1 with two lanes, 2.3 GB/s for G4, 5.7 GB/s for G5, before protocol overhead.

> **TRY IT**
>
> Go to G1 (`112211`), then:
>
> ```
> echo 3 > /proc/sys/vm/drop_caches     # every run, or the read comes from RAM
> dd if=/dev/block/sda of=/dev/null bs=1M count=1024
> ```
>
> At G1, anything above about 290 MB/s means the link isn't at G1, because that's physically impossible. Then run the same at max gear and compare. G1 is the clearest test: a simple `dd` may not fill a G3 link, so G3 and max gear can show similar numbers, while G1 is an unmistakable drop.

Reading the `state=` print

The debug print, `dev_err(hba->dev, "state=%d\n", hba->ufshcd_state)`, reads the state table in 2.3. `state=1` (OPERATIONAL) means all good. `state=3` (EH_SCHEDULED_FATAL) means MediaTek's link check failed and a reset is queued: the old failure is back, so look at what woke, or didn't wake, the W-LUN.

Test 3: without `echo 4`

This shows that MediaTek's lock isn't part of the fix.

> **TRY IT**
>
> ```
> W=/sys/bus/scsi/devices/0:0:0:49488/power/runtime_status
> # reboot, and don't touch clkscale_control
> cat $W                  # repeat until it says: suspended
> echo 112211 > /sys/kernel/debug/…/power_mode
> dmesg | grep -E "exit h8|ufs_mtk_config_pwr_mode"
> ```
>
> It passes if the `grep` prints nothing. No `exit h8` line means no failure. No `ufs_mtk_config_pwr_mode` line means MediaTek's own path never ran; if one does appear, something wrote `clkscale_control`. Then check the gear the same way as in the lane test.

3 Why it happened

3.1 Hibern8: the link's sleep, and who wakes it

What hibern8 is, physically. It is the deepest power state of the lanes that still keeps the link's settings. The M-PHY's phase-locked loop, which makes the gigabit bit clock, stops. The line drivers let go of the wires, and the receivers switch off their high-speed circuits; only a small wake-up detector stays on. Both ends go down together, because the host can't silence the chip's transmitter by itself. Coming out of hibern8 takes far less time than starting the link from scratch, because nothing has to be renegotiated.

Hibern8 is a state of the *link* only. Both controllers keep running, and the chip keeps VCC and VCCQ. Driven lines with termination draw current even with no data on them, which is why sleeping the link saves real power.

> **TWO MEANINGS**
>
> Hibern8 (say "hibernate") is not the PC kind of hibernate, where RAM is saved to disk. Only the link sleeps; the phone stays on.

Two ways in, and who brings the link back

- **Auto-hibern8 (the hardware does it).** The AHIT register (offset `0x18`) holds an idle time. When no command has arrived for that long, the controller takes the link down by itself. When a new command arrives (the doorbell), or when AHIT is set to 0, it brings the link back by itself. **FROM THE CODE** MediaTek sets 3.5 ms for Samsung chips, 2 ms for Micron and 1 ms for everyone else (`ufs_mtk_fix_ahit()`, `ufs-mediatek.c:1488`).
- **Software hibern8 (the driver does it).** The driver sends the UIC command `DME_HIBERNATE_ENTER`. The link then stays down until the driver sends `DME_HIBERNATE_EXIT`. In 6.6 that exit is sent by `ufshcd_uic_hibern8_exit()` (`ufshcd.c:4413`) and, on MediaTek, also by `ufs_mtk_link_set_hpm()` (`ufs-mediatek.c:2467`) inside its resume hook. Both run only when the W-LUN wakes (3.3).

> **PLAIN WORDS**
>
> Auto-hibern8 is a motion-sensor light: it goes off when nobody's around and comes back on by itself. Software hibern8 is a light switched off by hand: it stays off until someone switches it back on.

Why writing 0 to AHIT can't wake software hibern8

Auto-hibern8 is the controller's own small state machine, and it only undoes what it did itself. Software hibern8 was entered by a command, outside that state machine, so stopping the timer has nothing to undo. And on the board, by the time the link is in software hibern8, AHIT is already 0: MediaTek's suspend hook wrote 0 before the chip went to Sleep (3.3), and nothing writes it back until the W-LUN wakes. Writing 0 again doesn't even change the register.

Why there are two kinds

- **Auto-hibern8 covers the short gaps while the disk is in use.** After 1 ms of quiet the link sleeps, and the next doorbell wakes it, with no CPU, interrupt or driver code involved. Doing that in software would cost two UIC commands for every gap, each with an interrupt round trip.
- **Software hibern8 is for when more than the link goes down:** the chip into Sleep, the M-PHY powered off, clocks and supplies off. Before cutting power, software must know the link is in hibern8 (it waited for the command to finish) and that it will stay there. Auto-hibern8 can promise neither.

They take turns and never overlap. The timer is off whenever software drives the link, and comes back when software hands the link back:

- Suspend and resume: MediaTek's suspend hook turns it off (3.3), and the core writes it back as the last step of the W-LUN's wake (`ufshcd.c:9999`).
- A PMC: MediaTek's `pwr_change_notify` turns it off before and writes it back after (`ufs-mediatek.c:2023`).
- Clock gating: MediaTek lets the core enter software hibern8 during clock gating only when auto-hibern8 is disabled (`ufs-mediatek.c:1779`). On the board it isn't, so clock gating never uses software hibern8.

How the driver sees whether the link is awake

FROM THE CODE MediaTek's controller reports the link state in the top byte (bits 31 to 24) of register 0x14C (`ufs-mediatek.h:52`), and `ufs_mtk_wait_link_state()` (`ufs-mediatek.c:562`) polls it every 100 to 200 µs until it shows the wanted value or the time runs out:

VALUE	NAME IN THE CODE (`ufs-mediatek.h:134`)	MEANING
0	`VS_LINK_DISABLED`	Link not started
1	`VS_LINK_DOWN`	Link down
2	`VS_LINK_UP`	Awake and working
3	`VS_LINK_HIBERN8`	Asleep in hibern8
4	`VS_LINK_LOST`	Connection lost
5	`VS_LINK_CFG`	Being configured

```
static int ufs_mtk_wait_link_state(struct ufs_hba *hba, u32 state,
                                   unsigned long max_wait_ms)
{
        ...
        timeout = ktime_add_ms(ktime_get(), max_wait_ms);
        do {
                time_checked = ktime_get();
                if (host->legacy_ip_ver || host->ip_ver < IP_VER_MT6899) { ... older chips ... }
                else {
                        val = ufshcd_readl(hba, REG_UFS_UFS_MMIO_OTSD_CTRL);
                        val = val >> 24;
                }
                if (val == state)
                        return 0;
                usleep_range(100, 200);   /* Sleep for max. 200us */
        } while (ktime_before(time_checked, timeout));

        return -ETIMEDOUT;
}
```

ufs-mediatek.c:562 (trimmed)

MT6991 uses register 0x14C because its IP version is at least `IP_VER_MT6899`; older MediaTek chips read `REG_UFS_PROBE` instead. Bits 20 to 16 of the same register show the controller's internal state machine; MediaTek's idle wait uses them to spot a hibern8 entry or exit that's still in progress (values 8 to 13).

The core keeps its own record, `hba->uic_link_state`, but it changes only when software moves the link (`ufshcd.c:1905` in clock gating, `9608` in `ufshcd_link_state_transition()`). When the AHIT timer takes the link down, that variable still says ACTIVE. Auto-hibern8 is invisible to the core.

3.2 Runtime PM: who's who

Runtime PM is how Linux lets single parts of a running phone sleep while everything else keeps working. Every `struct device` has a usage counter and a state, active or suspended. `pm_runtime_get_sync(dev)` means "I need this device now": it adds 1 and, if the device sleeps, runs its wake-up function and waits. `pm_runtime_put()` means "I'm done"; at zero, the device may sleep after its autosuspend delay.

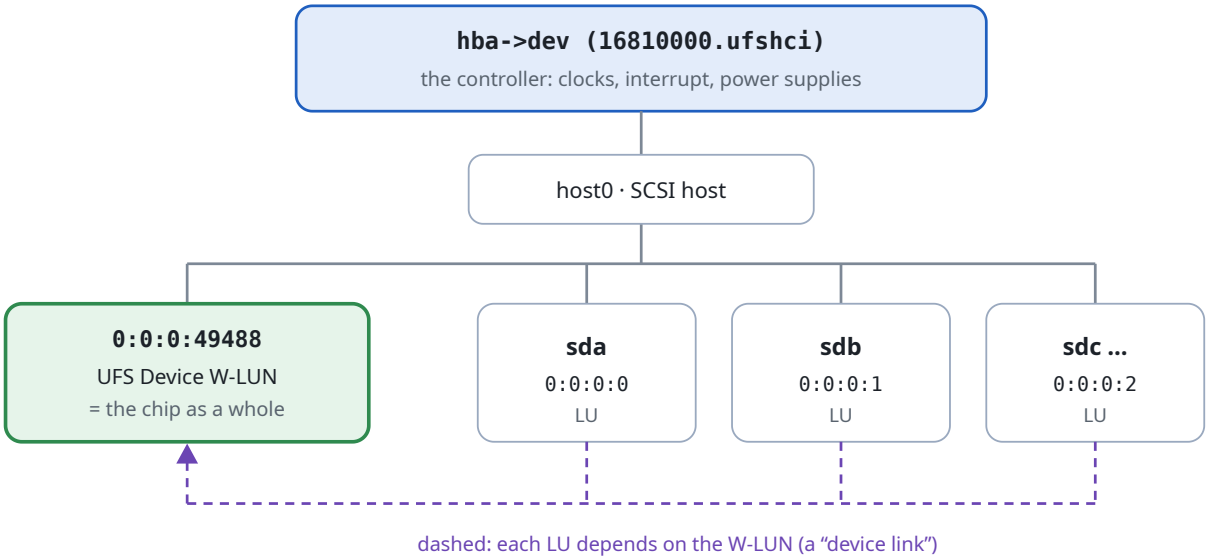


Figure 3. Solid lines are parent and child; dashed lines are “depends on”. The W-LUN is Linux's handle on the chip itself. There are two more W-LUNs (RPMB and BOOT) that don't matter here.

For UFS, Linux creates these objects:

- **The controller**, `hba->dev`: the platform device `16810000.ufshci`.
- **The SCSI host** under it, and under that the **LUs**: `sda`, `sdb` and so on, the readable and writable slices of the chip.
- **The UFS Device W-LUN** (well-known logical unit 0xD0, shown by Linux as `0:0:0:49488`). It stands for the whole chip. To put the chip to sleep or wake it, the driver sends START STOP UNIT to this address.

Four rules decide who wakes whom. Each has a physical reason:

1. **Waking a device wakes its parent first.** You can't reach the chip if the controller's clocks are off.
2. **Waking a parent never wakes its children.** Powering the controller doesn't mean anyone needs the chip.
3. **A parent can't sleep while a child is awake.**
4. **Device links add "depends on" arrows.** Each LU depends on the W-LUN (`device_link_add(…, DL_FLAG_PM_RUNTIME | DL_FLAG_RPM_ACTIVE)`, `ufshcd.c:5093`). So the W-LUN can't sleep while any LU is awake, and waking an LU wakes the W-LUN first.

FROM THE CODE On the board the LUs wait 500 ms of quiet before they sleep (`MTK_RPM_AUTOSUSPEND_DELAY_MS`, `ufs-mediatek.h:367`, set at `ufs-mediatek.c:1769`), and the W-LUN waits 0 ms (`ufshcd.c:10815`). So about half a second after the last disk access, the whole chip side is asleep. That is the half second from 1.2.

> **TWO MEANINGS**
>
> **Runtime suspend** (one device sleeps while the phone is on) is not **system suspend** (the whole phone sleeps). This bug is entirely about runtime suspend. And `ufshcd_rpm_get_sync()` versus `pm_runtime_get_sync()` is not two mechanisms: it's the same function given a different device (2.2).

3.3 What each sleep and wake actually does

The W-LUN going to sleep: `__ufshcd_wl_suspend()`

```
static int __ufshcd_wl_suspend(struct ufs_hba *hba, enum ufs_pm_op pm_op)
{
        ...
        ret = ufshcd_vops_suspend(hba, pm_op, PRE_CHANGE);   // 9830 MediaTek: AHIT off, link UP
        ...
                ret = ufshcd_set_dev_pwr_mode(hba, req_dev_pwr_mode);
                                              // 9840 START STOP UNIT: chip to Sleep
        ...
        ret = ufshcd_link_state_transition(hba, req_link_state, check_for_bkops);
                                              // 9860 DME_HIBERNATE_ENTER
        ...
        ret = ufshcd_vops_suspend(hba, pm_op, POST_CHANGE);  // 9879 MediaTek: link hardware off
        ...
}
```

ufshcd.c:9746 (trimmed)

FROM THE CODE The core does the middle two steps. The first and last are MediaTek's hooks:

- **Step 1, `PRE_CHANGE`: AHIT off, then wait for the link to be UP.** MediaTek's `ufs_mtk_auto_hibern8_disable()` (`ufs-mediatek.c:2997`) writes 0 to AHIT, waits up to 5 ms for the controller's state machine to go idle, then up to 100 ms for 0x14C to say UP. Three reasons: START STOP UNIT travels over the link, so the link must be UP to carry it; the core can't see auto-hibern8 (3.1), and checks that the link is active before starting (`ufshcd.c:9784`); and a 1 ms timer could take the link down in any gap between the steps. The function's failure branch says it was written for this path: "trigger error handler and break suspend".
- **Step 4, `POST_CHANGE`: switch off the host's link hardware** (`ufs-mediatek.c:2627` to `2655`). The chip is asleep and the link is in software hibern8, so nothing will use these until the next wake. All of it is on the SoC's side:
  1. UniPro into power-down: `ufs_mtk_link_set_lpm()` (`2521`) sets MediaTek's own attribute `VS_UNIPROPOWERDOWNCONTROL` (0xD0A8, `ufs-mediatek.h:128`) to 1.
  2. The M-PHY loses power: `phy_power_off()`, through `ufs_mtk_mphy_power_on()` (`607`).
  3. The controller's SRAM is switched off through an SMC, a call into MediaTek's secure firmware (`ufs_mtk_sram_pwr_ctrl()`; the macro is in mainline's `ufs-mediatek-sip.h`). Neither MediaTek's code nor mainline says what the SRAM holds.
  4. At G5 only, the clocks drop: `_ufs_mtk_clk_scale(hba, false)` (`2655`) moves `ufs_sel` to 273 MHz and the encryption clock to 416 MHz, and stops holding vcore at 0.65 V. This is the "G5 dip" that comes back in Part 4.

  A fifth action, holding the chip's reset line, runs only when the link is fully off, which never happens on the runtime path. If any step fails, the hook marks the link as off and returns `-EAGAIN`, which forces a full reset instead of a normal wake.

The W-LUN waking up: `__ufshcd_wl_resume()`

```
static int __ufshcd_wl_resume(struct ufs_hba *hba, enum ufs_pm_op pm_op)
{
        ...
        ret = ufshcd_vops_resume(hba, pm_op);               // 9939 MediaTek's resume hook
        ...
        if (ufshcd_is_link_hibern8(hba)) {
                ret = ufshcd_uic_hibern8_exit(hba);       // DME_HIBERNATE_EXIT
                ...
        }
        if (!ufshcd_is_ufs_dev_active(hba)) {
                ret = ufshcd_set_dev_pwr_mode(hba, UFS_ACTIVE_PWR_MODE);
                                          // START STOP UNIT: chip to Active
                ...
        }
        ...
        ufshcd_auto_hibern8_enable(hba);                    // 9999 AHIT timer back on
        ...
}
```

ufshcd.c:9927 (trimmed)

FROM THE CODE MediaTek's resume hook, `ufs_mtk_resume()` (`ufs-mediatek.c:2669`), undoes step 4 in reverse: SRAM on, M-PHY on, and, if the gear is G5, both clocks back to their fast sources with vcore raised first (`2699`). Then `ufs_mtk_link_set_hpm()` (`2467`) re-enables the controller (`ufshcd_hba_enable()`), takes UniPro out of power-down, sends `DME_HIBERNATE_EXIT` and sets up the controller's request queues again. That last step is a sign the controller really lost its working state while asleep.

The controller going to sleep and waking up

The controller sleeps right after the W-LUN (rule 3). `ufshcd_suspend()` turns the interrupt off, turns the clocks off (`ufshcd.c:10136`) and calls `ufshcd_vreg_set_lpm()` (`10147`) for the chip's supplies: the chip isn't Active, so VCC goes off; the link is in hibern8, so VCCQ goes into low-power mode. The voltage stays the same, but the regulator is told the load will be at most 1 mA, so the PMIC can switch it into a low-current mode. VCCQ stays on because the chip's controller has to keep its state.

MediaTek wraps this. Its `ufs_mtk_runtime_suspend()` (`ufs-mediatek.c:3615`) calls the core's version, puts more rails into low-power mode (`3626`), and then switches off the power domain the whole controller sits in (MTCMOS, `3629`; enabled by `mediatek,ufs-rtff-mtcmos`, `mt6991.dts:6764`). Waking runs the reverse:

```
static int ufshcd_resume(struct ufs_hba *hba)
{
        ...
        ufshcd_hba_vreg_set_hpm(hba);               // power supplies to full power
        ret = ufshcd_vreg_set_hpm(hba);             // 10171
        ...
        ret = ufshcd_setup_clocks(hba, true);       // 10176 clocks on
        ...
        ufshcd_enable_irq(hba);                     // interrupt on
        ...
}
```

ufshcd.c:10163 (trimmed)

That's all the controller's wake does: power, clocks, interrupt. Nothing about the link, nothing about the chip's mode.

Which vendor hook runs where

The hook names don't say whose suspend they mean. FROM THE CODE In 6.6, the vendor `suspend` and `resume` hooks are called by the W-LUN (`ufshcd.c:9830`, `9879`, `9939`). Up to Linux 5.13 the controller's own `ufshcd_suspend()` called them (`ufshcd_v5.13.c:8769`): the name stayed, and the caller moved together with the job (3.6). The controller's own sleep and wake give the vendor a different hook, `setup_clocks` (`ufshcd.c:9298`, `9329`), which also runs on every clock gate and ungate (`1910`, `1768`).

> **KEY POINT**
>
> In 6.6, the only wake-up that sends `DME_HIBERNATE_EXIT` and puts the chip back to Active is the W-LUN's. Waking the controller alone leaves the link asleep and the chip in Sleep.

3.4 The power ladder on the board

Hibern8 is only the first rung. This is everything that switches off as the board stays quiet, in order:

QUIET FOR	WHO ACTS	WHAT GOES DOWN	THE CHIP
1 ms	The AHIT timer, in hardware	The link goes into hibern8: both ends' transmitters and receivers.	Active, all supplies on
10 ms	Clock gating (`ufshcd_gate_work()`, `ufshcd.c:1910`) and MediaTek's `setup_clocks` hook (`ufs-mediatek.c:893`)	The controller's clocks. Once MediaTek reads the link as in hibern8 in 0x14C, also the chip's reference clock and the host's M-PHY.	Active, all supplies on
~500 ms	`__ufshcd_wl_suspend()` (`ufshcd.c:9746`)	AHIT off, the chip into Sleep, the link into software hibern8. Then the host's UniPro, M-PHY and SRAM, and at G5 the clocks.	Sleep, supplies on
right after	The controller's sleep: `ufshcd_suspend()` inside `ufs_mtk_runtime_suspend()`	Clocks off. Then the chip's VCC off and VCCQ into low-power mode. Then the controller's power domain.	Sleep, VCC off, VCCQ low-power

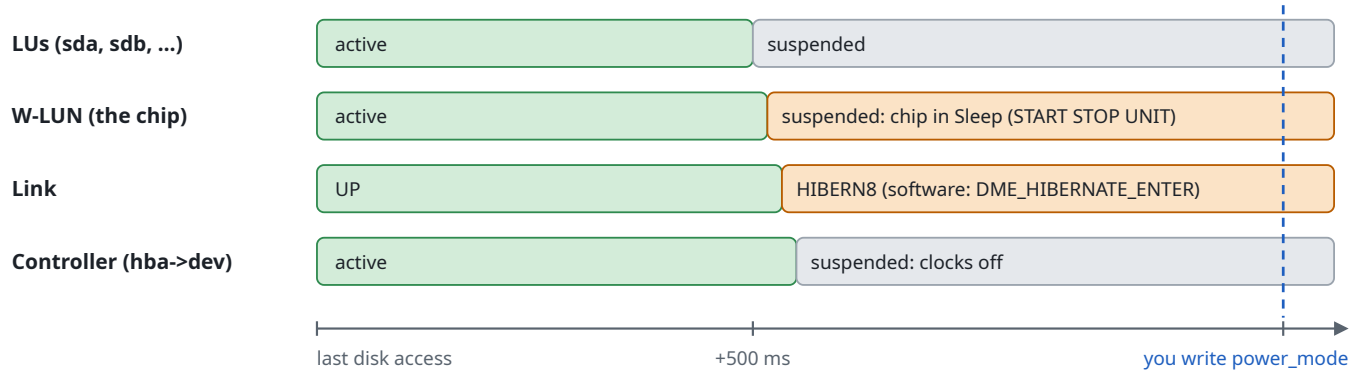


Figure 4. The sleep cascade. The LUs sleep after 500 ms of quiet; the W-LUN follows at once (chip to Sleep, link to software hibern8); then the controller turns its clocks off. By the time a command arrives, everything is asleep.

- **START STOP UNIT cuts no power.** It changes the chip's own mode: the chip finishes its work, stops using the flash, and becomes safe to lose VCC. Only the host switches the supplies, one rung later.
- **The 10 ms rung, in MediaTek's words:** "Gate ref-clk and poweroff mphy if link state is in OFF or Hibern8 by either Auto-Hibern8 or ufshcd_link_state_transition()". The reference clock is the steady clock the host supplies to the chip's side of the link; it may stop while the link is in hibern8. The next command ungates everything first, then the doorbell wakes the link.
- The figure above shows only "clocks off" for the controller. The controller's sleep also switches off the chip's VCC, puts VCCQ into low-power mode and powers off the controller's domain.
- **Waking climbs back down the ladder.** A normal read starts at the bottom and wakes everything (3.7). The old first line woke only the last rung (3.5).

> **KEY POINT**
>
> The AHIT timer does rung 1. The W-LUN's sleep does START STOP UNIT plus its own software hibern8. The chip's supplies are cut one rung later, by the controller's sleep.

3.5 The bug, row by row

When the command was typed, the board had been quiet for more than half a second, so every rung had already happened:

	BEFORE THE COMMAND	AFTER `pm_runtime_get_sync(hba->dev)`	AFTER `ufshcd_rpm_get_sync(hba)`
Controller	Power domain off, clocks off	On, clocks on	On, clocks on
Chip	Sleep, VCC off	VCC back on, **still in Sleep**	Active (START STOP UNIT)
Link	Software hibern8, AHIT = 0	**Still in software hibern8**	UP (`DME_HIBERNATE_EXIT`), AHIT back on

The old line names the controller, and by rule 2 waking a device never wakes its children. Nothing in the controller's wake sends a hibernate-exit or tells the chip to leave Sleep. Both are the W-LUN's job, and nobody woke the W-LUN. The same sleeping link then produced two different messages:

- **First version, timeout.** The PMC doesn't try to wake anything; it's a negotiation. Writing `PA_PWRMODE` made the host's UniPro send a power-change request, but in software hibern8 the wires carry nothing until a hibernate-exit, so the request never crossed and the chip never answered. The host had accepted the command (otherwise `uic error` would have appeared at once, `ufshcd.c:4229`); what never came was the end of the exchange, so after 500 ms the result was `completion timeout` (`4237`).
- **Second version, "exit h8 state fail".** MediaTek's `PRE_CHANGE` hook assumes the link is either UP or in auto-hibern8; in both cases writing AHIT = 0 ends with the link UP. Software hibern8 is the case it doesn't cover. AHIT was already 0, the link never came up, and after 100 ms the hook printed `exit h8 state fail` and scheduled a reset. The function's name says the same thing: `ufs_mtk_auto_hibern8_disable()` turns *auto*-hibern8 off; leaving software hibern8 isn't its job.
- **What happened after either failure.** The core marked the link broken and scheduled the error handler (`ufshcd.c:4270` to `4272`). The error handler wakes the W-LUN the right way, with `ufshcd_rpm_get_sync()` (`ufshcd.c:6381`), resets the controller and brings the link back at maximum gear. So the next disk access worked, but the requested gear was gone.

The code that complained

```
static int ufs_mtk_pwr_change_notify(struct ufs_hba *hba,
                                     enum ufs_notify_change_status stage,
                                     struct ufs_pa_layer_attr *dev_max_params,
                                     struct ufs_pa_layer_attr *dev_req_params)
{
        int ret = 0;
        static u32 reg;

        switch (stage) {
        case PRE_CHANGE:
                if (ufshcd_is_auto_hibern8_supported(hba)) {
                        reg = ufshcd_readl(hba, REG_AUTO_HIBERNATE_IDLE_TIMER);
                        ufs_mtk_auto_hibern8_disable(hba);        // return value ignored
                }
                ret = ufs_mtk_pre_pwr_change(hba, dev_max_params, dev_req_params);
                break;
        case POST_CHANGE:
                if (ufshcd_is_auto_hibern8_supported(hba))
                        ufshcd_writel(hba, reg, REG_AUTO_HIBERNATE_IDLE_TIMER);
                break;
        ...
}
```

ufs-mediatek.c:2023 (trimmed)

```
int ufs_mtk_auto_hibern8_disable(struct ufs_hba *hba)
{
        ...
        /* disable auto-hibern8 */
        ufshcd_writel(hba, 0, REG_AUTO_HIBERNATE_IDLE_TIMER);
        ...
        /* wait host return to idle state when auto-hibern8 off */
        ufs_mtk_wait_idle_state(hba, 5);

        ret = ufs_mtk_wait_link_state(hba, VS_LINK_UP, 100);
        if (ret) {
                dev_warn(hba->dev, "exit h8 state fail, ret=%d\n", ret);
                ...
                hba->force_reset = true;
                hba->ufshcd_state = UFSHCD_STATE_EH_SCHEDULED_FATAL;
                schedule_work(&hba->eh_work);
                ...
                /* trigger error handler and break suspend */
                ret = -EBUSY;
        }
        return ret;
}
```

ufs-mediatek.c:2997 (trimmed)

Two details matter. `pwr_change_notify` ignores the return value, so after a failed check the PMC still goes ahead. If the requested mode is the one the link already has, the core then returns success without sending anything (1.4), while a reset is queued: that's why the ported function now checks the state (2.3). And the AHIT value saved in `PRE_CHANGE` is kept in a function-wide `static`, shared by every caller.

> **KEY POINT: THE RULE THE FIX FOLLOWS**
>
> Whoever calls `ufshcd_config_pwr_mode()` must first hold the W-LUN awake (`ufshcd_rpm_get_sync()`) and keep the clocks on (`ufshcd_hold()`). Every caller in the kernel does: MediaTek's `clkscale_control` path (`ufs-mediatek.c:2322`), the error handler (`ufshcd.c:6381`), and probe and reset, which run their PMC on a freshly started link. Nothing enforces it: `ufshcd_config_pwr_mode()` doesn't check. That's why the old code compiled and ran, and only failed in the hardware.

3.6 Why the original line was right in Linux 4.19

Was this "decided in 6.6"? It wasn't. It changed in Linux 5.14, in 2021, and 6.6 inherited it. The ported code comes from Qualcomm's Linux 4.19, from before the change.

Up to Linux 5.13: one switch did everything

FROM THE CODE Before 5.14, waking `hba->dev` ran `ufshcd_runtime_resume()`, which called `ufshcd_resume(hba, UFS_RUNTIME_PM)`. In v5.13 that single function did all of it:

```
static int ufshcd_resume(struct ufs_hba *hba, enum ufs_pm_op pm_op)
{
        ...
        ret = ufshcd_setup_clocks(hba, true);                   // 8860 ← clocks on
        ...
        ufshcd_enable_irq(hba);                                 // 8865 ← interrupt on
        --------- 6.6 cuts the function here ---------
        ret = ufshcd_vops_resume(hba, pm_op);                   // 8872 ← vendor hook
        ...
        if (ufshcd_is_link_hibern8(hba)) {
                ret = ufshcd_uic_hibern8_exit(hba);             // 8880 ← DME_HIBERNATE_EXIT
        ...
        if (!ufshcd_is_ufs_dev_active(hba)) {
                ret = ufshcd_set_dev_pwr_mode(hba, UFS_ACTIVE_PWR_MODE);
                                                                // 8905 ← chip to Active
        ...
        ufshcd_auto_hibern8_enable(hba);                        // 8928 ← AHIT timer back on
        ...
}
```

upstream v5.13, drivers/scsi/ufs/ufshcd.c, in sources/ as ufshcd_v5.13.c:8846 (trimmed; numbers are line numbers)

So in 4.19, `pm_runtime_get_sync(hba->dev)` woke the controller, the link and the chip. Qualcomm's msm-4.19 debugfs function starts with exactly that line (`ufs-debugfs.c:1211` in msm-4.19, copied in file 04), and the ported code is a port of that function. It was correct when it was written.

The deadlock that moved the job

The change is commit `b294ff3e3449`, "scsi: ufs: core: Enable power management for wlun", by Asutosh Das and Can Guo (both at Qualcomm). It was committed on 10 May 2021 and first released in Linux 5.14. Its commit message explains what went wrong before it:

1. When the controller went to sleep, its own sleep function sent START STOP UNIT to the chip, to put the chip to sleep first.
2. But by rule 3, the controller only sleeps after all its children are asleep, and that includes the SCSI devices. Their request queues were already suspended.
3. A block-layer change, `52abca64fd94` ("scsi: block: Do not accept any requests while suspended"), made any request to a suspended queue wait until the queue resumes.
4. So the START STOP UNIT sat waiting for a queue that would never resume, and the controller's sleep waited for the command. Nothing could move. The commit message shows the stuck call chain: `ufshcd_runtime_suspend → ufshcd_suspend → ufshcd_set_dev_pwr_mode → __scsi_execute → blk_get_request → blk_queue_enter → schedule`.

The fix gave the W-LUN its own runtime PM, and moved the chip-and-link work into *its* sleep and wake functions. With the device links (rule 4), the W-LUN sleeps after all the LUs and wakes after the controller, so its START STOP UNIT always has a working path.

The proof: the old function, cut in two

Put v5.13's `ufshcd_resume()` next to 6.6 and cut it right after `ufshcd_enable_irq()`. The top half is 6.6's `ufshcd_resume()`, the controller's wake-up. The bottom half is 6.6's `__ufshcd_wl_resume()`, the W-LUN's wake-up. The lines are nearly identical, as you can check against the excerpts in 3.3.

STEP	V5.13: ALL IN `ufshcd_resume()`, RUN WHEN `hba->dev` WAKES	6.6: WHERE IT MOVED
Power supplies to full power	yes	`ufshcd_resume()`, the controller
Clocks on	line 8860	`ufshcd_resume()`, the controller
Interrupt on	line 8865	`ufshcd_resume()`, the controller
Vendor resume hook	line 8872	`__ufshcd_wl_resume()`, the W-LUN
`DME_HIBERNATE_EXIT`	line 8880	`__ufshcd_wl_resume()`, the W-LUN
Chip to Active (START STOP UNIT)	line 8905	`__ufshcd_wl_resume()`, the W-LUN
AHIT timer back on	line 8928	`__ufshcd_wl_resume()`, the W-LUN

LINUX 4.19 TO 5.13

`pm_runtime_get_sync(hba->dev)` wakes the controller, the link and the chip.

LINUX 5.14 AND LATER, INCLUDING 6.6

`pm_runtime_get_sync(hba->dev)` wakes the controller only. The link and the chip need the W-LUN.

PLAIN WORDS

In the old house, the main switch also turned on the hallway light. In 2021 the house was rewired and the hallway got its own switch. The ported instructions were written for the old house: "flip the main switch". Flipping it left the hallway dark.

3.7 Why normal disk use never hit it

Normal reads and writes come in from the bottom of the tree, and runtime PM wakes everything they depend on:

1. A read on `/dev/sda` goes to that LU's queue, and the block layer wakes the LU before sending the command.
2. The LU depends on the W-LUN through the device link (`ufshcd.c:5093`), so the W-LUN wakes first: hibernate-exit, chip to Active.
3. Before that, the W-LUN's parents bring the controller up.

So every read wakes the controller, the chip and the link before its command goes out. The debugfs path came in from the top, at the controller, and waking a device never wakes anything below it.

3.8 Linux words, spec words

"Suspend" and "resume" are Linux's words, from its power-management core. The specifications name the same states differently:

WHAT	NAME IN THE SPEC	WHICH SPEC
The chip's power mode	Active, Sleep, PowerDown, DeepSleep, set with START STOP UNIT	JEDEC UFS (JESD220). SSU itself is a SCSI command; UFS defines the power-mode values it carries.
The link's sleep state	HIBERN8 is the M-PHY line state. UniPro calls it "Hibernate" and uses `DME_HIBERNATE_ENTER` and `EXIT`.	MIPI M-PHY and UniPro, not JEDEC
The hardware timer	Auto-Hibernate, the AHIT register	JEDEC UFSHCI (JESD223)

FROM THE CODE The core translates with a table, `ufs_pm_lvl_states[]` (`ufshcd.c:243`). Each Linux power level is a pair: a chip power mode and a link state. The board's runtime level, 3, is {Sleep, hibern8} (line 247), so "Linux suspends the W-LUN" means START STOP UNIT to Sleep plus the link into hibern8. The numbers carry straight through: `UFS_SLEEP_PWR_MODE` is 2, JEDEC's value for Sleep, and the core puts it in the command's power-condition field (`pwr_mode << 4`, `ufshcd.c:9516`).

4 What we have now

4.1 The fix holds

SEEN ON THE BOARD Since the one-line change: no more `exit h8 state fail`, lane changes from 2 to 1 work, and fio speeds follow the gear that was set. Everything below is a separate question that only shows up at G5: how fast it goes, and why the numbers wander.

4.2 What the board has shown so far

SEEN ON THE BOARD Every measurement from 31 October and 1 November in one place. The rest of Part 4 explains them.

#	WHAT WAS DONE	WHAT WAS SEEN
1	Writes to `power_mode` with the fix, any gear	No failures; lane changes 2 → 1 pass (31 Oct)
2	fio at different gears	Speeds follow the gear
3	`clk_summary` while idle, before any test (12:11)	`ck_sel` at 273 MHz, under `ck_mainpll_d4_d2`
4	G1 → G4 through `power_mode`	`ck_sel` rises to 499.2 MHz, and stays there at G4
5	A G5 write, then reading the clock while idle; then dd at G5	Idle: `suspended`, 273 MHz. During dd: `active`, 499.2 MHz. Both as predicted
6	A 4 GB fio run at G5, with another I/O running in parallel to keep the disk awake	About 3200 MB/s without SQPOLL, 3800 with
7	Sequential writes, 4 GB, iodepth 64, io_uring, G5L2 alternating with G1L2	Average 3495, maximum 3720, minimum 2926 MB/s; with SQPOLL still 3000 to 3600. G4 and G1, at 2 lanes and at 1, are steady
8	Same writes to `/data`, with the prep script, `power/control = on` and clock gating off, 10 iterations	A steady ~3000
9	`clk_summary` for the encryption clock during those runs (14:07)	`ck_aes_ufsfde_sel` at 416 MHz, under `ck_univpll_d6`, the slow source
10	After a reboot: no prep, `power/control = auto`, then `echo 5; echo 4`	Writes ~3000; the encryption clock hops between 416 and 546 MHz
11	WriteBooster, as configured on the board	About 40 GB available (as configured); it doesn't refill while the board is idle

#	WHAT WAS DONE	WHAT WAS SEEN
12	Boot logs	The board boots at G4. Which log line said so isn't identified yet

4.3 Clock scaling, from scratch

What it is. Clock scaling switches the UFS path between two speed settings, and it moves two knobs together: the controller's clock `ufs_sel` (273 or 499.2 MHz, with vcore raised for the fast one) and the link's gear (the top gear, or a lower one). The reason is power: every clock tick charges and discharges millions of transistor gates, and the fast clock also needs a higher voltage, whose cost grows with the square of the voltage. An idle disk on the slow setting saves a lot.

FROM THE CODE There are two kinds:

- **The core's automatic scaling** uses Linux's devfreq with the simple_ondemand policy. Every polling period it measures what fraction of the time commands were outstanding (`ufshcd.c:1535` to `1539`), and scales up above a threshold and down below another. MediaTek's settings for boards that use it: poll every 200 ms, up above 40% busy, down below 20%, and the lower gear is G4 (`ufs-mediatek.c:3032` to `3036`). Going down it drops the gear first, then the clock; going up, the clock first, then the gear (`ufshcd_devfreq_scale()`, `ufshcd.c:1371`), so the clock is never slower than the gear needs.
- **MediaTek's manual scaling**, through `clkscale_control`. The device tree has `mediatek,ufs-clkscale-only-by-user` (`mt6991.dts:6768`): "only by user" means only when user space asks, by writing that file. "User" here is the kernel's word for anything outside the kernel, PowerHAL included. MediaTek reads the flag into its own capability bit (`ufs-mediatek.c:826`) and then doesn't turn on the core's scaling (`1763`), so the core never calls `ufshcd_devfreq_init()` (`ufshcd.c:8763`). There is no governor on the board.

"Off" means "fixed", not "highest". With no governor, the clock stays wherever something last put it. On the board, three things put it somewhere: the core at boot, MediaTek's helper (4.4), and the ported block (4.10).

> **TRY IT: IS AUTOMATIC SCALING ON?**
>
> ```
> ls /sys/class/devfreq/                      # is 16810000.ufshci listed?
> ls /sys/bus/platform/devices/16810000.ufshci/clkscale_enable
> ```
>
> Both appear only when the core's automatic scaling is on (`ufshcd_add_lus()`, `ufshcd.c:8763` to `8774`). On the board, expect neither. The `clkscale/` directory doesn't count as evidence: MediaTek creates it at controller init (`ufs-mediatek.c:1787`), from the device-tree flag alone.

The values of `clkscale_control`

FROM THE CODE `clkscale_control_store()` (`ufs-mediatek-sysfs.c:710`). The store ignores every value if the chip's spec version is below UFS 4.0; the board's chip runs G5, so it's 4.0.

VALUE	HELP TEXT	WHAT IT DOES ON THE BOARD
0	free run	Refused: there is no governor to hand control back to (`ufs-mediatek-sysfs.c:736`)

VALUE	HELP TEXT	WHAT IT DOES ON THE BOARD
1	scale down	Gear to G4, then the encryption clock to 416, `ufs_sel` to 273, vcore released
2	scale up	vcore raised, `ufs_sel` to 499.2, the encryption clock to 546, then the gear to the maximum. 2 is also what `cat` shows after boot
3	scale down and forbid change	Like 1, then locked
4	scale up and forbid change	Like 2, then locked
5	allow change and free run	Clears the lock. Nothing else on the board

The gear changes keep the lane count and mode: MediaTek's power mode change copies `hba->pwr_info` and changes only the gear (`ufs-mediatek.c:2280`). MediaTek never chooses G2 or G3 itself. The only place it runs HS-G1 is its eye-monitor and scrambler debug code (`ufs-mediatek-sysfs.c:307` and `614`).

Two checks at the top: the lock and the memory

Every value from 1 to 4 ends up in `ufs_mtk_dynamic_clock_scaling()` (`ufs-mediatek.c:2364`), which can return without doing anything:

1. **The lock**, `clk_scale_forbid` (line 2376). Writing 3 or 4 sets it, and while it's set, 1 to 4 are ignored. Writing 5 clears it. It always starts cleared after a reboot.
2. **The memory**, `scale_mode` (lines 2380 to 2382), a static variable holding the last request. A request equal to the last one is skipped.

So 5 then 4 is enough. 5 clears the lock. And whenever `scale_mode` already says "up", the clocks are already fast while the disk is awake: the only thing that slows them later without touching `scale_mode` is MediaTek's G5 sleep hook, and its G5 wake undoes that before any command runs. When the clocks are slow while the disk is awake, `scale_mode` says "free run" or "down", so a 4 runs.

> **WATCH OUT: TWO TRAPS IN THIS FILE**
>
> - **`cat` can lie.** The store records the value you wrote even when the lock or the memory made it do nothing. Check the clocks themselves (4.11), not `cat`. The same quirk is a free detector: after an `echo 4`, if `cat` later shows anything else, something else on the system writes this file. The store accepts a `powerhal_set:` prefix, so PowerHAL is the obvious suspect, but that's an inference from the prefix, not something we've seen.
> - **Don't write to `upthreshold` or `downdifferential`**, the two files next to it. They tune the governor that doesn't exist here, and both store functions call `mutex_lock(&hba->devfreq->lock)` without checking for NULL. `hba->devfreq` is NULL on the board, so writing a number is a kernel oops. Reading them is safe.

4.4 The two clocks, and who moves them

FROM THE CODE

	`ufs_sel` (`ck_sel`)	ENCRYPTION CLOCK `ufs_fde` (`ck_aes_ufsfde_sel`)
Sources	273 MHz `ck_mainpll_d4_d2` / 499.2 MHz `ck_univpll_d5`	416 MHz `ck_univpll_d6` / 546 MHz `ck_mainpll_d4`
At boot	The core sets 499.2 (`ufshcd_init_clocks()`, `ufshcd.c:9398`)	**Nothing sets it.** It stays on whatever source the bootloader left
MediaTek's helper `_ufs_mtk_clk_scale()` (`ufs-mediatek.c:3039`)	Yes	Yes: the only code that ever sets its source (lines 3091 and 3101)
...which runs on	`clkscale_control` 1 to 4; the G5 sleep and wake hooks (`2655`, `2699`); a boot fixup that only applies to a 26 MHz reference clock or a UFS 3.x chip (`1517`)	
The core's `clk_set_rate()` (the ported block)	Yes: it has a frequency entry, `<273000000 499200000>` (`mt6991.dts:6732`)	Never: its entry is `<0 0>`

MediaTek's helper always moves vcore and both clocks together, in an order that keeps them safe:

	FIRST	THEN	LAST
Up	vcore floor to 0.65 V	`ufs_sel` to 499.2 MHz	Encryption clock to 546 MHz
Down	Encryption clock to 416 MHz	`ufs_sel` to 273 MHz	vcore floor released

It switches each mux between its two sources with `clk_set_parent()`. Its comment explains why it avoids `clk_set_rate()`: the rates in the device tree must match the sources, so that the core's own `clk_set_rate()` finds the rate already set and does nothing, and this "prevent[s] chaning rate of pll clock that is shared between modules" (`ufs-mediatek.c:3140`).

> KEY POINT
>
> Only MediaTek's helper moves the encryption clock. A gear change through `power_mode` never does, and the helper's wake hook only acts when the disk wakes up already at G5. Switching from G1 or G4 into G5 leaves the encryption clock wherever it was.

4.5 The encryption engine

Encryption on Android has a software side and a hardware side. The hardware sits in the controller, not in the UFS chip.

- **Software.** fscrypt, inside f2fs, decides which key and which per-block number each block of a file uses; Android's vold and keystore supply the keys. fscrypt can do the AES maths on the CPU, or hand it to hardware through blk-crypto. The `inlinecrypt` mount option chooses the hardware.
- **Hardware.** FROM THE CODE An AES engine inside the UFS host controller, defined by the UFSHCI specification. MediaTek enables it with `hba->caps |= UFSHCD_CAP_CRYPTO` (`ufs-mediatek.c:1757`) and switches it on through its secure firmware (`ufs_mtk_crypto_enable()`, line 258). If that call fails, MediaTek turns the capability off again (line 266), and the CPU would have to do the encryption instead. The driver loads keys into the engine's key slots and tags each request with a slot number; the engine encrypts the data as it streams out to the chip. Its clock is `ufs_fde`, "FDE" for full-disk encryption.
- **The chip only ever stores ciphertext.** Reads of the raw device, such as `dd if=/dev/block/sda`, bypass the engine entirely.

TRY IT

```
mount | grep " /data "                 # inlinecrypt in the options: the engine does the work
dmesg | grep "crypto enable failed"    # nothing here: MediaTek's engine is on
```

4.6 Why only G5 varies: the five-part chain

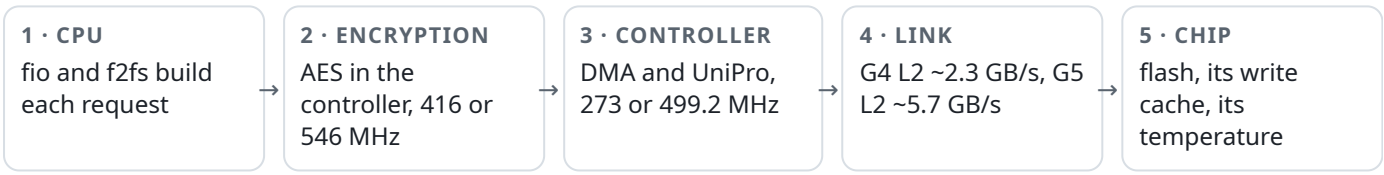


Figure 5. Every write to `/data` passes through all five. The slowest one sets the speed. Up to G4 that's the link; at G5 it's one of the others.

At G4 with two lanes, the link tops out around 2.3 GB/s. That's below what the other four can do, so every run hits the same wall, and the G4 and G1 numbers are steady. G5 roughly doubles the link, to about 5.7 GB/s before overhead. UFS 4.0 chips are rated somewhere around 3 to 4 GB/s for sequential writes, depending on the vendor. So at G5, for the first time, the wire stops being the limit, and the next-slowest part shows through. Each of the other four can vary between runs:

- **The CPU:** its frequency, and which core the fio thread lands on (4.8).
- **The encryption engine:** which source its clock is on (4.4 and 4.7).
- **The controller:** `ufs_sel` is at 499.2 MHz whenever the disk is busy at G5 (row 5 of 4.2), so it's probably not the variable one.
- **The chip:** how full its fast write cache is, background work, temperature (4.9).

4.7 The most coherent story so far

NOT YET PROVEN This story fits every row of 4.2, but it hasn't been tested. 4.11 is the test.

1. **After boot, something scales the disk down.** A write of 1 to `clkscale_control` does exactly what was found: G4, `ufs_sel` at 273 MHz, the encryption clock at 416 MHz. That would also explain the "boots at G4"

log, because `PMC Success! … gear=[4, 4]` is printed only by MediaTek's own `clkscale_control` path. The likely writer is PowerHAL.

2. **The switch to G5 doesn't touch the encryption clock.** The disk wakes while still at G4, so MediaTek's wake hook does nothing, and the ported block raises only `ufs_sel`. G5 then runs with encryption at 416 MHz, about 3000 MB/s.
3. **With `power/control = auto`, timing decides.** Once the disk sleeps and wakes at G5, MediaTek's hooks put the encryption clock on 546 MHz. Whether a run got the fast clock depends on whether that just happened. That's the 3000 to 3700 mix of row 7.
4. **With `power/control = on`, it stays stuck.** No sleeps, no wakes: the encryption clock stays at 416 MHz for every run. That's the steady 3000 of rows 8 and 9.

The numbers fit, which is suggestive but not proof: 416/546 = 0.76, and the worst run divided by the best is 2926/3720 = 0.79.

An alternative for step 1: nothing scaled the disk down, and the encryption clock simply sat on the bootloader's choice of source, because nothing sets it at boot (4.4). The fix would be the same.

Row 10 is a different case. After that reboot, `echo 4` put the encryption clock on 546 MHz, and working through the sequence, it should have stayed there during every G5 run. So that 3000 probably had a different cause: with no prep script, the CPU governor was back to its default, and the CPU can be the limit (row 6). Same number, different limit, which is why it all felt inconsistent.

> **TRY IT: CHECK STEP 1**
>
> ```
> dmesg | grep "PMC Success"            # a gear=[4, 4] line soon after boot: something scaled down
> cat /sys/bus/platform/devices/16810000.ufshci/clkscale/clkscale_control
>                                       # right after boot: current: 1 or 3 means someone wrote it
> dmesg | grep "query device ref clk"   # "= 1" would mean a 26 MHz reference clock
> cat /sys/bus/platform/devices/16810000.ufshci/device_descriptor/specification_version
> ```
>
> The last two rule out the boot fixup. G5 working already argues against both: with a 26 MHz reference clock MediaTek caps the host at G4 (`ufs-mediatek.c:1920`), and a UFS 3.x chip can't do G5 at all.

4.8 fio, the CPU, and a clean benchmark setup

SQPOLL

With `sqthread_poll=1`, a kernel thread on another CPU picks up new requests by polling, so the fio thread doesn't spend a system call on every batch. Without it, one fio thread does all the submitting and all the reaping of completions. At G5 the disk finishes a request faster than that one thread can hand it the next one, so the queue runs dry. Row 6's 600 MB/s gap was time the disk spent waiting for fio.

- To confirm: fio prints `cpu: usr=…, sys=…` for each job. If they add up to nearly 100% without SQPOLL, that thread is the limit.
- Other ways to the same result: bigger blocks (`bs=1m`, fewer requests per GB), a deeper queue, `numjobs=2`, or fio pinned to a big core (`cpus_allowed`).
- SQPOLL is a workaround for a host-side limit. The Qualcomm SoC runs probably didn't need it because the disk was the slow part there: at G4 or below the link caps everything near 2 GB/s, long before the CPU

matters. Whatever you choose, keep the fio settings identical on both platforms.

The settings that remove noise

- **The prep script, every session.** `echo 0 0 0 > /proc/sys/kernel/printk` stops kernel messages from going to the serial console, which is very slow and can stall a CPU for milliseconds; `dmesg` still records everything. Turning tracing off removes per-event recording. The `performance` governor keeps the CPUs at full speed, which matters most here, because the CPU can be the limit at G5.
- **`power/control = on` for the W-LUN while hunting.** The chip side never sleeps: no wake-up cost at the start of a run, no G5 clock dip, and the chip can flush its write cache between runs. Phones run with `auto`, so once the cause is known, also measure with `auto` and report both.
- **Clock gating off is optional.** It only acts after 10 ms with nothing in flight, which never happens during a streaming run. Turning it off removes one more variable and costs only idle power (`clkgate_enable`; writing 0 makes the core hold the clocks permanently).
- **No keep-alive I/O.** A running benchmark keeps the disk awake by itself; the clock only drops after 500 ms with no I/O at all. A second job shares the link with the benchmark, so the number would depend on what the other job is doing.
- **Long enough runs.** 4 GB at about 3.3 GB/s is only about 1.2 seconds, so startup is a noticeable share. Use bigger runs, but for writes keep each one below the free WriteBooster space, and start every run from the same cache state (4.9).

TRY IT: TWO CLEAN FIO LINES

```
# reads of the raw device: no filesystem, no encryption; --readonly makes fio refuse to write
fio --name=g5read --filename=/dev/block/sda --readonly --rw=read --direct=1 \
    --ioengine=io_uring --sqthread_poll=1 --bs=1m --iodepth=32 \
    --runtime=20 --time_based --ramp_time=2

# writes to a file in /data: f2fs and the encryption engine are in the path
fio --name=g5write --filename=/data/local/tmp/fio.bin --rw=write --size=8g \
    --direct=1 --ioengine=io_uring --sqthread_poll=1 --bs=1m --iodepth=64 \
    --end_fsync=1
```

Never point a write job at the raw device: it would overwrite the partitions on it.

4.9 WriteBooster: the chip's fast write cache

WriteBooster is a region of the chip's flash run in a fast mode (one bit per cell). Writes land there first, and later the chip **flushes** them into normal, denser flash, freeing the space again. While the cache has room, writes are fast; when it fills, writes go straight to the slower flash.

FROM THE CODE Why the WriteBooster cache doesn't refill while the board is idle:

- The chip needs VCC to flush. The core says so in a comment in `ufshcd_wb_need_flush()`: "The ufs device needs the vcc to be ON to flush".
- At 500 ms idle the W-LUN sleeps, the chip goes to Sleep and VCC goes off (3.4), unless the core decides a flush is needed. In that case it keeps the chip Active and VCC on during the sleep (`ufshcd.c:9820` to `9825`).

- "Needed" means 10% or less of the cache available in user-space-reduction mode (`ufshcd.c:6189`), or below `wb_flush_threshold`, 40% by default, in preserve-user-space mode (`ufshcd.c:9086`).
- With about 40 GB free the chip is above both, so the chip sleeps instead of flushing. Nothing we changed touches this.

TRY IT: READ THE CACHE STATE

```
U=/sys/bus/platform/devices/16810000.ufshci
cat $U/wb_on $U/enable_wb_buf_flush $U/wb_flush_threshold
cat $U/flags/wb_enable $U/flags/wb_flush_en $U/flags/wb_flush_during_h8
cat $U/attributes/wb_avail_buf $U/attributes/wb_cur_buf $U/attributes/wb_flush_status
```

`wb_avail_buf` counts in tenths: 10 means 100% free, 4 means 40%. Read it before and after one 4 GB write. If it never drops, WriteBooster may not be in use at all (`wb_on` or `wb_enable` at 0), and the writes are going straight to the slower flash. To let the chip flush between runs, keep `power/control = on` and wait: with the chip powered and the link only in auto-hibern8, it can flush if `wb_flush_during_h8` is 1, and `wb_avail_buf` should climb back.

4.10 The ported clock-scaling block on the board

FROM THE CODE The block decides "scale up" for any gear above G1 and calls `ufshcd_scale_clks()`, around the PMC. That function has two halves. MediaTek's half, which raises vcore first and then switches sources, returns at once because core scaling is off (`ufs_mtk_clk_scale_notify()`, `ufs-mediatek.c:3172`). The core's half, `ufshcd_set_clk_freq()` (`ufshcd.c:1066`), calls `clk_set_rate()` on every clock with a frequency entry, which on the board means `ufs_sel` only, and skips it when the core's record, `clki->curr_freq`, already matches.

SEEN ON THE BOARD A G1 visit sets `ufs_sel` to 273 MHz. The next gear-up sets it to 499.2 MHz. At G4 it then stays at 499.2 MHz, because MediaTek's sleep and wake hooks only touch the clocks at G5, and nothing else lowers it (rows 3 and 4 of 4.2).

Is 499.2 MHz at G4 unsafe? It depends on history, because the block never touches vcore:

- If the disk last woke at G5, MediaTek's resume hook asked for 0.65 V, and nothing at G1 to G4 releases that. Then 499.2 MHz at G4 has its voltage, and the only cost is wasted power: MediaTek runs G4 at 273 MHz.
- If MediaTek's scale-down ran since then, the 0.65 V request was released, and 499.2 MHz runs without it, a pairing MediaTek's own code never allows.

TRY IT: IS THE VCORE FLOOR HELD?

```
grep -i ufshci /sys/kernel/debug/regulator/regulator_summary
```

On the controller's `dvfsrc-vcore` line, `650mV 2147483mV` means the floor is held; `0mV …` means it's released.

Why delete it. For G2 to G5 it either does nothing or undoes its own G1 change. Its one real action, `clk_set_rate(ufs_sel, 273 MHz)` at G1, is the call MediaTek's design avoids (4.4): the clock framework picks the source by itself, which MediaTek's flow never exercises. It can't move the encryption clock, which is the one that

matters for G5 writes to `/data`. And dropping it costs nothing: a slow link with the fast controller clock is the normal state during every boot, because the core sets 499.2 MHz before the link starts. File 07 drops it.

> **PLAIN WORDS**
>
> "Don't gear changes need clock scaling?" Only if something can lower the clock. The rule only works one way: the controller's clock must never be *slower* than the gear needs, and faster is always fine. On the board three things lower it: this block (at G1), `clkscale_control` 1 or 3, and MediaTek's own sleep at G5, which its wake undoes. The block creates the very need it then fills.

4.11 The decisive run

One run with everything fixed separates the encryption clock from the CPU:

> **TRY IT**
>
> ```
> sh mtk_prepare.sh
> echo on > /sys/bus/scsi/devices/0:0:0:49488/power/control
> echo 0  > /sys/bus/platform/devices/16810000.ufshci/clkgate_enable
> C=/sys/bus/platform/devices/16810000.ufshci/clkscale/clkscale_control
> echo 5 > $C; echo 4 > $C
> grep -w -B8 ufs_fde /sys/kernel/debug/clk/clk_summary   # walk up to ck_aes_ufsfde_sel
> ```
>
> The encryption clock must read 546000000, under `ck_mainpll_d4`, every time you look: with the chip side never sleeping, there is no more hopping. Append these lines to `mtk_prepare.sh`, so a reboot doesn't lose them. Then run the 10 G5L2 ↔ G1L2 iterations, and once, read the encryption clock in the middle of a write (fio in the background, wait a second, then the `grep`).

RESULT	MEANING	NEXT
About 3600 or more, steady	The slow encryption clock was the limit in rows 8 and 9; the CPU probably was in row 10	Pick a permanent fix (4.12)
Still about 3000, with the clock steady at 546	The encryption engine isn't the limit	f2fs or the chip: compare reading a large file in `/data` with reading the raw device at G5, and look at the WriteBooster numbers before each run
The clock isn't at 546 after `echo 4`	The write didn't take effect	Check `cat $C`, the lock, and dmesg for MediaTek's messages

While you're at it, read the core's error counters before and after a batch of G5 runs: `cat /sys/kernel/debug/ufshcd/16810000.ufshci/stats`. If "PHY Adapter Layer errors" or "Data Link Layer errors" grow at G5 but not at G4, the link itself is losing frames and resending them, which also costs throughput without ever showing an error.

4.12 Permanent fixes, if the encryption clock is confirmed

- **A boot step: `echo 5`, then `echo 4`, to `clkscale_control`.** It puts vcore and both clocks on their fast settings and locks out later scale-downs. It holds even with `power/control = auto`: a G5 sleep drops the clocks, but the G5 wake restores them before any command, and gear changes through `power_mode` never lower the encryption clock. The simplest fix, and no rebuild.
- **A change in MediaTek's vendor module.** The clean one: when a power mode change targets G5, call `_ufs_mtk_clk_scale(hba, true)` first, the way MediaTek's own `clkscale_control` path does before its PMC to the maximum gear. A smaller alternative: run the helper once at init, so the encryption clock never sits on the bootloader's choice. Either needs a vendor-module rebuild.
- **The vendor-side route.** File 02, `ufs_mtk_config_gear_lane()`, already calls `_ufs_mtk_clk_scale(hba, true)` before a G5 change. It was never wired in, because the debugfs file lives in the core.

The core can't do it by itself: `_ufs_mtk_clk_scale()` is private to MediaTek's module, and the core doesn't even know the encryption clock has two speeds.

4.13 Where a patch could go

- **The board's own vendor tree:** the realistic first step, once file 07 is built and tested.
- **Mainline Linux:** possible, but it would be a new feature. Mainline 6.18 has no way to change gear or lanes at runtime, only the read-only `power_info/gear` and `lane` files. A patch would need to be written against current mainline, not 6.6; to use a core helper like Part A of file 07 instead of exporting internals; and to explain what happens on boards where the governor would undo the gear.
- **The Android common kernel:** takes it only after it's upstream.
- **MediaTek:** two small, concrete reports. The `upthreshold` and `downdifferential` NULL-pointer crash, in a vendor file mainline doesn't have (fix: return `-EOPNOTSUPP` when `hba->devfreq` is NULL). And, if 4.11 confirms it, the encryption clock left on its slow source until the first scale-up, even though MediaTek's init comments that the default is the maximum frequency.

4.14 Open items

Everything still open, from both days, in rough order:

1. **Run the decisive run** (4.11), and read the encryption clock once in the middle of a write.
2. **Check step 1 of the story** in 4.7: `dmesg | grep "PMC Success"` and `cat clkscale_control` right after boot.
3. **Build and test file 07** (2.5). It also makes the write handler return the real error instead of `cnt`, so a failed `echo` really fails.
4. **Move or delete the two debug dumps** that can run with the clocks off (2.4).
5. **Make `echo 5`, then `echo 4` a boot step**, and watch the `current:` line to see whether anything else writes `clkscale_control` (4.3).
6. **Collect the remaining numbers:** `mount | grep " /data "` for `inlinecrypt` (4.5), the WriteBooster values before and after a write (4.9), and the error counters before and after a batch of G5 runs (4.11).
7. **Never write `upthreshold` or `downdifferential`** on this board (4.3).
8. **To go back to max gear** after other changes, write it through `power_mode` (`552211`). A `clkscale_control` 2 or 4 is skipped if MediaTek's memory already says "up" (4.3).

9. **If the old failure ever comes back,** print register 0x14C inside `ufs_mtk_auto_hibern8_disable()` at the moment it fails (needs a vendor-module rebuild). Bits 31 to 24 reading 3 would confirm a link stuck in hibern8.
10. **The board's vendor tree** and the OnePlus copy have the same vendor code, but check line numbers before relying on them.
11. `ufsdbg_cfg_pwr_param()` **writes** `hba->max_pwr_info.info.pwr_rx = FAST_MODE`**.** That's core state the error handler uses to restore the link. It's harmless today, because the chip already reports FAST there; don't write anything else into it.
12. **Optional:** trace the PHY framework's reference count, to settle whether a controller-only wake powers the M-PHY back on (Part 5).
13. **Then decide** the permanent fix (4.12) and where the patch goes (4.13).

5 Corrections along the way

These statements changed during the investigation; knowing which helps in weighing the rest. Over the two days:

WHAT WAS SAID	WHAT'S ACTUALLY TRUE
Force every request to FAST mode.	Only the AUTO modes need mapping. Forcing FAST breaks SLOW (PWM) requests (2.3).
First: the ported clock lines are dangerous. Then: they're harmless because MediaTek's hook does nothing.	MediaTek's hook does nothing, but the core still calls `clk_set_rate()` on `ufs_sel`. On the board the block raises the clock at every gear-up after a G1 visit, and at G4 nothing lowers it again (4.10).
`down_write(&hba->lock)` is a problem.	That lock doesn't exist in 6.6; the equivalent is `clk_scaling_lock`.
Put the gear and lane function in MediaTek's vendor module (files 02, 03 and the patch).	The debugfs file is in the core, which can't call vendor code. The core versions (05, 06, 07) are the ones to use. The vendor route matters again only for a permanent clock fix (4.12).
The link-state register was called `REG_UFS_PROBE`.	On MT6991 it's `REG_UFS_UFS_MMIO_OTSD_CTRL`, 0x14C. `REG_UFS_PROBE` belongs to older MediaTek chips.
"Nobody sends `DME_HIBERNATE_EXIT`."	Imprecise. The W-LUN's wake sends it. It just never ran, because nothing woke the W-LUN.
"Since 5.14…", which read as if 6.6 had decided it.	It changed in 5.14 (2021); 6.6 inherited it.
`echo 4` presented as if it were part of the fix.	The fix works without it. `echo 4` is a lock against other writers, and it puts both clocks on their fast sources (4.3).
Earlier: after a controller-only wake, "the M-PHY stayed powered off".	Not shown. The controller's wake runs MediaTek's `setup_clocks` hook, which calls `phy_power_on()`; whether the PHY really came back depends on the PHY framework's reference count, which hasn't been traced. The explanation doesn't need it: software hibern8 keeps the wires silent until a hibernate-exit either way.
"the ported block raises the clock without the vcore."	It depends on history. If the disk last woke at G5, the 0.65 V request is still held. Only after a MediaTek scale-down is it released (4.10).
The G4 boot explained by a 26 MHz reference clock or a UFS 3.x chip, with consequences drawn from that (`echo 4` ignored, the clock parked for good).	Unlikely, because G5 works on the board, and neither case allows G5. A scale-down after boot is the likelier story, still unproven (4.7).

WHAT WAS SAID	WHAT'S ACTUALLY TRUE
"Write 5, then 1, then 4."	5 then 4 is enough. A re-check showed: whenever MediaTek's memory says "up", the clocks are already fast while the disk is awake (4.3).
"Within half a second of any disk access … the link was UP."	The W-LUN was awake, but after 1 ms of quiet the link was usually in *auto*-hibern8.
"Android's PowerHAL writes this file."	An inference from the `powerhal_set:` prefix, not something we've seen. The `current:` detector will tell (4.3).
A row 6 result described as reads.	That run wasn't recorded as reads or writes, so 4.2 doesn't say either.
Line numbers `ufs-mediatek.c:825` and `2626`.	One off each: 826 and 2627.
Earlier: MediaTek's boot fixup at `ufs-mediatek.c:1516`.	It starts at line 1517.
The sequence was assumed to come straight from Qualcomm.	It came through a port for the MediaTek SoC, which took it from the Qualcomm SoC kernel.
An early file 05 used a variable named `pwr`.	The parameter is `desired_pwr_mode`. Fixed in 05 and 06.

KEY POINT

The original bug wasn't in any one file. Each piece was right on its own terms: Qualcomm's line for Linux 4.19, the core's 2021 change, MediaTek's auto-hibern8 check for the case it was written for. The bug lived in the gaps between them. The G5 question looks the same so far: the encryption clock is managed by one piece of code (MediaTek's helper) and silently skipped by another (gear changes through the core).

6 Command cheat sheet

Every board command from this document, in one place. Set these once per shell:

```
U=/sys/bus/platform/devices/16810000.ufshci                # the controller
W=/sys/bus/scsi/devices/0:0:0:49488                        # the UFS Device W-LUN
C=$U/clkscale/clkscale_control                             # MediaTek's clock control
P=/sys/kernel/debug/ufshcd/16810000.ufshci/power_mode      # the debugfs file (adjust)
D=/sys/kernel/debug/ufshcd/16810000.ufshci                 # the core's debugfs directory
K=/sys/kernel/debug/clk
```

Read the state

```
cat $P                                       # gear, lanes, mode the core recorded
cat $W/power/runtime_status                  # active or suspended (the chip side)
cat $W/power/control                         # auto or on
grep -w -B8 ufs_sel $K/clk_summary           # walk up to ck_sel: 273000000 or 499200000
grep -w -B8 ufs_fde $K/clk_summary           # walk up to ck_aes_ufsfde_sel: 416000000 or 546000000
cat $K/ck_sel/clk_rate $K/ck_sel/clk_parent
cat $K/ck_aes_ufsfde_sel/clk_rate $K/ck_aes_ufsfde_sel/clk_parent
grep -i ufshci /sys/kernel/debug/regulator/regulator_summary   # vcore request: 650mV or 0mV
cat $D/stats                                 # error counters (PA, DL, resets ...)
cat $C                                       # current: N (can lie, see 4.3)
cat $U/clkgate_enable $U/clkgate_delay_ms    # clock gating
cat $U/attributes/wb_avail_buf $U/attributes/wb_cur_buf $U/attributes/wb_flush_status
cat $U/flags/wb_enable $U/flags/wb_flush_en $U/flags/wb_flush_during_h8
cat $U/wb_on $U/enable_wb_buf_flush $U/wb_flush_threshold
cat $U/device_descriptor/specification_version
ls /sys/class/devfreq/; ls $U/clkscale_enable  # automatic scaling present?
mount | grep " /data "                       # inlinecrypt?
dmesg | grep -E "exit h8|PMC Success|err_handler started|query device ref clk|crypto enable failed"
```

Change settings

```
sh mtk_prepare.sh                    # printk quiet, tracing off, performance governor
echo on   > $W/power/control         # chip side never sleeps   (auto = default)
echo 0    > $U/clkgate_enable        # clock gating off          (1 = default)
echo 5 > $C; echo 4 > $C             # both clocks fast, vcore up, max gear, locked
echo 552211 > $P                     # G5, 2 lanes, FAST   (112211 = G1, 442211 = G4)
```

Benchmark

```
echo 3 > /proc/sys/vm/drop_caches     # before buffered tests
fio --name=g5read --filename=/dev/block/sda --readonly --rw=read --direct=1 \
    --ioengine=io_uring --sqthread_poll=1 --bs=1m --iodepth=32 \
    --runtime=20 --time_based --ramp_time=2
fio --name=g5write --filename=/data/local/tmp/fio.bin --rw=write --size=8g \
    --direct=1 --ioengine=io_uring --sqthread_poll=1 --bs=1m --iodepth=64 --end_fsync=1
dd if=/dev/block/sda of=/dev/null bs=1M count=8192    # quick raw read
```

WATCH OUT

- Never write to the raw device (`/dev/block/sda`): it holds your partitions. Use `--readonly` for raw reads, and a file in `/data` for writes.
- Never write to `$U/clkscale/upthreshold` or `downdifferential` on this board (4.3).
- A reboot resets `power/control` , clock gating, the CPU governor and MediaTek's lock. Keep them in `mtk_prepare.sh` .

7 Glossary

ADAPT

UniPro's equalization training: when the link enters G4 or G5, both ends tune their transmitters and receivers to the wire. MediaTek asks for the full version, `PA_INITIAL_ADAPT`, on every change into those gears.

AH8 (auto-hibern8)

The controller takes the link into hibern8 by itself after an idle time, and brings it back by itself on the next command or when AHIT is set to 0.

AHIT

The controller register (offset 0x18) holding the auto-hibern8 idle time. 0 means off. 1 ms on the board.

blk-crypto

The block layer's interface to inline encryption hardware: requests carry a key and a per-block number that the driver passes to the engine.

`clk_scale_forbid`

MediaTek's lock, set by `clkscale_control` 3 and 4 and cleared by 5. While it's set, MediaTek's own clock and gear changes are skipped.

`clkscale_control`

MediaTek's sysfs file for manual clock scaling, values 0 to 5 (4.3).

Clock gating

Switching the controller's clocks off after a short idle time (10 ms on MediaTek) and back on before the next command. Separate from runtime PM and from clock scaling.

Clock scaling

Moving the controller's clock and the link's gear between a fast and a slow setting, to save power. Automatic (devfreq) or manual (`clkscale_control`). Only the manual kind exists on the board.

`curr_freq`

The core's record of each scalable clock's rate. MediaTek's own clock moves don't update it, which is how the core's record and the real clock can disagree.

Device link

A "depends on" arrow between two Linux devices. Each LU depends on the W-LUN, so waking an LU wakes the W-LUN first.

devfreq

Linux's framework for changing a device's frequency according to load. It drives the core's automatic UFS clock scaling. Never started on the board.

DME

The part of UniPro that takes management commands: `DME_GET`, `DME_SET`, `DME_HIBERNATE_ENTER`, `DME_HIBERNATE_EXIT`.

Doorbell

Controller register (offset 0x58) with one bit per command slot.

EH (error handler)

The core's recovery work. It resets the controller, restarts the link and restores the maximum gear.

f2fs

The flash-friendly filesystem Android uses for `/data`.

FDE, `ufs_fde`

The clock of the controller's inline AES engine (`ck_aes_ufsfde_sel`), 416 or 546 MHz. "FDE" stands for full-disk encryption.

fscrypt

The kernel's file-based encryption, used by Android for `/data`. It can encrypt on the CPU or through inline hardware.

Gear

Bits per second per lane. HS-G1 to G5 are the fast family, each double the last.

GKI

Google's Generic Kernel Image, the common kernel that holds the UFS core.

`hba`

The core's `struct ufs_hba`, describing one controller. `hba->dev` is the controller's `struct device`.

Hibern8

The link's deepest sleep that keeps its settings. Entered by the AHIT timer (auto) or by a command (software).

`host_sem`

The core's semaphore that keeps user requests, error handling, system suspend and shutdown from overlapping.

`inlinecrypt`

The mount option that makes fscrypt use the inline encryption hardware.

Lane

One pair of wires carrying data in one direction.

LU

Logical unit: a readable, writable slice of the chip (`sda`, `sdb`, ...).

M-PHY

The electrical layer: the wires, their voltages, and recovering bits and timing from them.

MTCMOS

MediaTek's name for a switchable power domain inside the SoC. MediaTek switches off the UFS controller's domain after the controller sleeps.

Mux

A clock multiplexer: it outputs one of several source clocks. `ck_sel` and `ck_aes_ufsfde_sel` are muxes with two sources each.

Only by user

The device-tree flag `mediatek,ufs-clkscale-only-by-user`: clock scaling happens only when user space writes `clkscale_control`.

PACP

Control frames sent by UniPro's lowest layer. A PMC is a `PACP_PWR_req` from the host and a `PACP_PWR_cnf` back from the chip.

PMC

Power mode change: switching gear, lanes or mode.

PMIC

The power-management chip (MT6363 here) whose regulators make the board's supply rails.

Power level (`rpm_lvl`)

The core's pairing of a chip power mode and a link state, from `ufs_pm_lvl_states[]`. The board's runtime level 3 is {Sleep, hibern8}.

PowerHAL

Android's power service. It may write `clkscale_control` with a `powerhal_set:` prefix; not confirmed on the build.

Rate A, rate B

The two base frequencies for HS gears. MediaTek uses B.

Reference clock

The steady clock the host supplies to the chip's side of the link. MediaTek stops it once the link is in hibern8.

Runtime PM

Linux's per-device sleep and wake while the system is running.

`scale_mode`

MediaTek's static memory of its last clock-scaling request. A repeat of the same request is skipped.

`setup_clocks`

The vendor hook the core calls whenever it turns the controller's clocks on or off.

SMC

Secure monitor call: asks the secure firmware to do something Linux can't do directly. MediaTek uses SMCs for the UFS SRAM and the encryption engine.

SQPOLL

An io_uring mode (fio `sqthread_poll=1`) where a kernel thread picks up new requests by polling, saving the application a system call per batch.

SRAM

On-chip memory. The UFS controller has a block of it, switched off when the chip side sleeps.

SSU (START STOP UNIT)

The SCSI command that sets the chip's power mode: Active, Sleep, PowerDown, DeepSleep. It cuts no power by itself.

Symbol clock

A clock running at the link's own rate: `unipro_tx_sym` for sending, and one receive clock per lane.

UIC command

A command for the link layer, written to controller register 0x90.

`uic_link_state`

The core's record of the link state. Only software moves update it; auto-hibern8 doesn't.

UniPro

The link layer: frames, acknowledgements, resending, speed and power state.

UniPro power-down

MediaTek's attribute `VS_UNIPROPOWERDOWNCONTROL` (0xD0A8), set to 1 when the chip side sleeps, so the host's UniPro engine stops.

UPIU

The packet format that carries SCSI commands over UFS.

VCC

The supply rail for the chip's flash. The host switches it off once the chip is in Sleep.

VCCQ

The 1.2 V supply rail for the chip's controller and its end of the link. Low-power mode during runtime sleep, never off.

vcore

The SoC's core supply voltage. The 499.2 MHz `ufs_sel` source needs at least 0.65 V.

vops

The table of vendor hooks the core calls (`pwr_change_notify`, `suspend`, `resume`, `setup_clocks`, …).

W-LUN

Well-known logical unit. The UFS Device W-LUN (0xD0, shown by Linux as `0:0:0:49488`) stands for the whole chip.

WriteBooster

The chip's fast write cache, flushed later into normal flash. It needs VCC to flush.

8 Files and sources

`~/Personal_Study/kernel/mt6991_ufs_gear_lane/`

FILE	WHAT IT IS
`UFS_MT6991_bug_fix_and_G5.html` and `.pdf`	This document.
`UFS_gear_lane_fix_explained.html` and `.pdf`	The earlier write-up, with the 1 November additions marked.
`UFS_gear_lane_fix_explained_v1.html` and `_v1.pdf`	Its first version, from 31 October, unchanged.
`01_mtk_reference.c`	MediaTek's own `ufs_mtk_config_pwr_mode()`, with its steps marked.
`02_ufs_mtk_config_gear_lane.c`, `03_gear_lane_sysfs.c`, `0001-ufs-mediatek-add-gear_lane-sysfs.patch`	A vendor-module gear and lane function, a sysfs file for it, and both as a patch. Not compiled. Relevant again for a permanent clock fix (4.12).
`vendor_6.6/`, `patched_6.6/`	The three MediaTek files before and after that patch.
`04_qcom_msm-4.19_ufsdbg_config_pwr_mode.c`	Qualcomm's original, which was ported to the MediaTek SoC.
`05_core_ufs-debugfs_config_pwr_mode.c`	An early core version built on `ufs_debugfs_get_user_access()`.
`06_ported_ufsdbg_config_pwr_mode_fixed.c`	The ported function, with the fixes marked `CHANGED`. This is what runs on the board.
`07_clean_ufsdbg_config_pwr_mode.c`	The clean version (2.5). Not compiled yet.
`sources/`	The source copies this document cites: `android15-6.6/` (`ufshcd.c`, `ufshcd.h`, `ufshcd-priv.h`, `ufs-debugfs.c`), `mtk_vendor_6.6/` (the MediaTek files and `mt6991.dts`), `upstream_v5.13_v5.14/`, `qcom_msm-4.19/`, and a `README.md` that maps citations to files.

Which copy a citation means: `ufshcd.c`, `ufshcd.h`, `ufshcd-priv.h` and `ufs-debugfs.c:68` are the 6.6 core; `ufshcd_v5.13.c` is upstream v5.13; `ufs-debugfs.c:1211` is Qualcomm's; `ufs-mediatek*.c`, `ufs-mediatek.h` and `mt6991.dts` are MediaTek's.

9 References

Commits

- `b294ff3e3449`: "scsi: ufs: core: Enable power management for wlun". Asutosh Das and Can Guo; merged for Linux 5.14 (2021). Gave the W-LUN its own runtime PM and moved the chip-and-link work there.
- `52abca64fd94`: "scsi: block: Do not accept any requests while suspended". The block-layer change that turned the old design into a deadlock.
- `7dec5cedf7b2`: "scsi: ufs: core: Serialize AHIT register access between sysfs and host driver". Related; doesn't change this story.
- `f29ec85ac4fd`: "scsi: ufs: host: mediatek: Return error directly on idle wait timeout". Related; doesn't change this story.

Source copies read

- Android common kernel `android15-6.6`: `drivers/ufs/core/ufshcd.c`, `ufs-debugfs.c`, `ufshcd-priv.h`, `include/ufs/ufshcd.h`.
- MediaTek vendor code, OnePlus's public MT6991 tree, `kernel_device_modules-6.6`: `ufs-mediatek.c`, `ufs-mediatek.h`, `ufs-mediatek-sysfs.c` and the other files in `sources/mtk_vendor_6.6/`, plus `mt6991.dts`.
- Upstream Linux v5.13 and v5.14: `drivers/scsi/ufs/ufshcd.c`.
- Qualcomm msm-4.19 (LineageOS sm8250 kernel): `drivers/scsi/ufs/ufs-debugfs.c`.
- Mainline Linux, the 6.18 tree in `~/Personal_Study/kernel/linux`: `drivers/ufs/host/ufs-mediatek-sip.h` (the SMC behind `ufs_mtk_sram_pwr_ctrl()`), `include/ufs/ufs.h` (`UFS_VREG_LPM_LOAD_UA`, 1000 µA), `drivers/clk/clk.c:3285` (how `clk_summary` lays out its lines), `drivers/devfreq/devfreq.c:882` (devfreq devices take their parent's name).
- The ported code in `ufs-debugfs.c` (31 October) and the board screenshots and measurements (31 October and 1 November).

Specifications named

- JEDEC JESD220, Universal Flash Storage (the chip, its power modes, WriteBooster).
- JEDEC JESD223, UFS Host Controller Interface (the controller, AHIT, inline encryption).
- MIPI UniPro and MIPI M-PHY (the link, hibern8, ADAPT, PACP).

Written with Claude Code, 1 November 2026, from two days of debugging with Shrihari. Line numbers refer to the source copies in `sources/`. Labels: "Seen on the board" means it was measured on the board; "From the code" means it was read in those copies; "Not yet proven" means it hasn't been tested.